

CS-471L Mini Compiler Project Report

Group: 2023CS32, 2023CS36

Week 16 Final Deliverable

1 Introduction

This project implements a mini compiler front end for a compact MicroJava subset. The completed system includes grammar analysis, lexical analysis, top-down parsing, bottom-up LR parsing, semantic analysis through a symbol table, centralized error handling, integration through a single dispatcher executable, and final demonstration outputs.

2 Grammar Specification

The source language supports classes, fields, arrays, methods, parameters, local declarations, assignment, calls, conditionals, loops, returns, printing, arithmetic expressions, relational expressions, character constants, and boolean constants.

2.1 Original and LL(1) Grammar

```
# Section 1 Grammar

This project uses a compact MicroJava subset with classes, fields, methods, local declarations,
assignments, calls, loops, conditionals, returns, printing, arrays, arithmetic, relational
expressions, and character constants.

## Original BNF

The original grammar keeps expression precedence left-recursive for the LR parser.

- 'Program ::= class id { MemberList }'
- 'MemberList ::= Member MemberList | epsilon'
- 'Member ::= Type id MemberAfter'
- 'MemberAfter ::= ; | [ number ] ; | ( ParamListOpt ) Block'
- 'Type ::= int | char | boolean | void'
- 'ParamListOpt ::= ParamList | epsilon'
- 'ParamList ::= Type id ParamListTail'
- 'ParamListTail ::= , Type id ParamListTail | epsilon'
- 'Block ::= { StmtList }'
- 'StmtList ::= Stmt StmtList | epsilon'
- 'Stmt ::= Type id VarDeclAfter ; | id StmtIdTail ; | return ExprOpt ; | if ( Expr ) Stmt | while ( Expr
    ) Stmt | Block ; | print ( Expr ) ;'
- 'VarDeclAfter ::= [ number ] | epsilon'
- 'StmtIdTail ::= AssignTarget = Expr | ( ArgListOpt )'
- 'AssignTarget ::= [ Expr ] | epsilon'
- 'ExprOpt ::= Expr | epsilon'
- 'ArgListOpt ::= ArgList | epsilon'
- 'ArgList ::= Expr ArgListTail'
- 'ArgListTail ::= , Expr ArgListTail | epsilon'
- 'RelOp ::= == | != | < | <= | > | >='
- 'AddOp ::= + | -'
- 'MulOp ::= * | / | %'
```

```

- 'Factor ::= id FactorAfter | number | char_const | true | false | ( Expr )'
- 'FactorAfter ::= ( ArgListOpt ) | [ Expr ] | epsilon'
- 'Expr ::= Expr RelOp AddExpr | AddExpr'
- 'AddExpr ::= AddExpr AddOp Term | Term'
- 'Term ::= Term MulOp Factor | Factor'

## LL(1) Transformed Grammar

- 'Program ::= class id { MemberList }'
- 'MemberList ::= Member MemberList | epsilon'
- 'Member ::= Type id MemberAfter'
- 'MemberAfter ::= ; | [ number ] ; | ( ParamListOpt ) Block'
- 'Type ::= int | char | boolean | void'
- 'ParamListOpt ::= ParamList | epsilon'
- 'ParamList ::= Type id ParamListTail'
- 'ParamListTail ::= , Type id ParamListTail | epsilon'
- 'Block ::= { StmtList }'
- 'StmtList ::= Stmt StmtList | epsilon'
- 'Stmt ::= Type id VarDeclAfter ; | id StmtIdTail ; | return ExprOpt ; | if ( Expr ) Stmt | while ( Expr
    ) Stmt | Block | ; | print ( Expr ) ;'
- 'VarDeclAfter ::= [ number ] | epsilon'
- 'StmtIdTail ::= AssignTarget = Expr | ( ArgListOpt )'
- 'AssignTarget ::= [ Expr ] | epsilon'
- 'ExprOpt ::= Expr | epsilon'
- 'ArgListOpt ::= ArgList | epsilon'
- 'ArgList ::= Expr ArgListTail'
- 'ArgListTail ::= , Expr ArgListTail | epsilon'
- 'Expr ::= RelExpr'
- 'RelExpr ::= AddExpr RelExprTail'
- 'RelExprTail ::= RelOp AddExpr | epsilon'
- 'RelOp ::= == | != | < | <= | > | >='
- 'AddExpr ::= Term AddExprTail'
- 'AddExprTail ::= AddOp Term AddExprTail | epsilon'
- 'AddOp ::= + | -'
- 'Term ::= Factor TermTail'
- 'TermTail ::= MulOp Factor TermTail | epsilon'
- 'MulOp ::= * | / | %'
- 'Factor ::= id FactorAfter | number | char_const | true | false | ( Expr )'
- 'FactorAfter ::= ( ArgListOpt ) | [ Expr ] | epsilon'

Transformation notes:
- Direct left recursion in expressions was replaced by 'AddExprTail', 'TermTail', and related tail non-
  terminals.
- Common prefixes such as member declarations and statement forms were left-factored.
- Empty productions are written as 'epsilon'.

```

3 FIRST and FOLLOW Sets

```

# FIRST and FOLLOW Sets

| Non-terminal | FIRST | FOLLOW |
|---|---|---|
| 'Program' | '{ class }' | '{ $ }' |
| 'MemberList' | '{ boolean, char, epsilon, int, void }' | '{ }' |
| 'Member' | '{ boolean, char, int, void }' | '{ boolean, char, int, void, }' |
| 'MemberAfter' | '{ (, ;, [ }' | '{ boolean, char, int, void, }' |
| 'Type' | '{ boolean, char, int, void }' | '{ id }' |
| 'ParamListOpt' | '{ boolean, char, epsilon, int, void }' | '{ ) }' |
| 'ParamList' | '{ boolean, char, int, void }' | '{ ) }' |
| 'ParamListTail' | '{ ,, epsilon }' | '{ ) }' |
| 'Block' | '{ { }' | '{ ;, boolean, char, id, if, int, print, return, void, while, {, } }' |
| 'StmtList' | '{ ;, boolean, char, epsilon, id, if, int, print, return, void, while, { }' | '{ } }' |
| 'Stmt' | '{ ;, boolean, char, id, if, int, print, return, void, while, { }' | '{ ;, boolean, char, id,
    if, int, print, return, void, while, {, } }' |
| 'VarDeclAfter' | '{ [, epsilon }' | '{ ; }' |
| 'StmtIdTail' | '{ (, =, [ }' | '{ ; }' |
| 'AssignTarget' | '{ [, epsilon }' | '{ = }' |
| 'ExprOpt' | '{ (, char_const, epsilon, false, id, number, true }' | '{ ; }' |
| 'ArgListOpt' | '{ (, char_const, epsilon, false, id, number, true }' | '{ ) }' |

```

```

| 'ArgList' | '{ (, char_const, false, id, number, true }' | '{ ) }' |
| 'ArgListTail' | '{ ,, epsilon }' | '{ ) }' |
| 'Expr' | '{ (, char_const, false, id, number, true }' | '{ ), ,, ;, ] }' |
| 'RelExpr' | '{ (, char_const, false, id, number, true }' | '{ ), ,, ;, ] }' |
| 'RelExprTail' | '{ !=, <, <=, ==, >, >=, epsilon }' | '{ ), ,, ;, ] }' |
| 'RelOp' | '{ !=, <, <=, ==, >, >= }' | '{ (, char_const, false, id, number, true }' |
| 'AddExpr' | '{ (, char_const, false, id, number, true }' | '{ !=, ), ,, ;, <, <=, ==, >, >=, ] }' |
| 'AddExprTail' | '{ +, -, epsilon }' | '{ !=, ), ,, ;, <, <=, ==, >, >=, ] }' |
| 'AddOp' | '{ +, - }' | '{ (, char_const, false, id, number, true }' |
| 'Term' | '{ (, char_const, false, id, number, true }' | '{ !=, ), +, ,, -, ;, <, <=, ==, >, >=, ] }' |
| 'TermTail' | '{ %, *, /, epsilon }' | '{ !=, ), +, ,, -, ;, <, <=, ==, >, >=, ] }' |
| 'MulOp' | '{ %, *, / }' | '{ (, char_const, false, id, number, true }' |
| 'Factor' | '{ (, char_const, false, id, number, true }' | '{ !=, %, ), *, +, ,, -, /, ;, <, <=, ==, >, >=, ] }' |
| 'FactorAfter' | '{ (, [, epsilon }' | '{ !=, %, ), *, +, ,, -, /, ;, <, <=, ==, >, >=, ] }' |

```

4 LL(1) Predictive Parsing Table

The non-recursive predictive parser uses the transformed non-left-recursive grammar and an explicit stack. Its parsing table is included below.

```

# LL(1) Parsing Table

| Non-terminal | Lookahead | Production |
|---|---|---|
| 'AddExpr' | '(' | 'AddExpr -> Term AddExprTail' |
| 'AddExpr' | 'char_const' | 'AddExpr -> Term AddExprTail' |
| 'AddExpr' | 'false' | 'AddExpr -> Term AddExprTail' |
| 'AddExpr' | 'id' | 'AddExpr -> Term AddExprTail' |
| 'AddExpr' | 'number' | 'AddExpr -> Term AddExprTail' |
| 'AddExpr' | 'true' | 'AddExpr -> Term AddExprTail' |
| 'AddExprTail' | '!=' | 'AddExprTail -> epsilon' |
| 'AddExprTail' | ')' | 'AddExprTail -> epsilon' |
| 'AddExprTail' | '+' | 'AddExprTail -> AddOp Term AddExprTail' |
| 'AddExprTail' | ',' | 'AddExprTail -> epsilon' |
| 'AddExprTail' | '-' | 'AddExprTail -> AddOp Term AddExprTail' |
| 'AddExprTail' | ';' | 'AddExprTail -> epsilon' |
| 'AddExprTail' | '<' | 'AddExprTail -> epsilon' |
| 'AddExprTail' | '<=' | 'AddExprTail -> epsilon' |
| 'AddExprTail' | '==' | 'AddExprTail -> epsilon' |
| 'AddExprTail' | '>' | 'AddExprTail -> epsilon' |
| 'AddExprTail' | '>=' | 'AddExprTail -> epsilon' |
| 'AddExprTail' | ']' | 'AddExprTail -> epsilon' |
| 'AddOp' | '+' | 'AddOp -> +' |
| 'AddOp' | '-' | 'AddOp -> -' |
| 'ArgList' | '(' | 'ArgList -> Expr ArgListTail' |
| 'ArgList' | 'char_const' | 'ArgList -> Expr ArgListTail' |
| 'ArgList' | 'false' | 'ArgList -> Expr ArgListTail' |
| 'ArgList' | 'id' | 'ArgList -> Expr ArgListTail' |
| 'ArgList' | 'number' | 'ArgList -> Expr ArgListTail' |
| 'ArgList' | 'true' | 'ArgList -> Expr ArgListTail' |
| 'ArgListOpt' | '(' | 'ArgListOpt -> ArgList' |
| 'ArgListOpt' | ')' | 'ArgListOpt -> epsilon' |
| 'ArgListOpt' | 'char_const' | 'ArgListOpt -> ArgList' |
| 'ArgListOpt' | 'false' | 'ArgListOpt -> ArgList' |
| 'ArgListOpt' | 'id' | 'ArgListOpt -> ArgList' |
| 'ArgListOpt' | 'number' | 'ArgListOpt -> ArgList' |
| 'ArgListOpt' | 'true' | 'ArgListOpt -> ArgList' |
| 'ArgListTail' | ')' | 'ArgListTail -> epsilon' |
| 'ArgListTail' | ',' | 'ArgListTail -> , Expr ArgListTail' |
| 'AssignTarget' | '=' | 'AssignTarget -> epsilon' |
| 'AssignTarget' | '[' | 'AssignTarget -> [ Expr ]' |
| 'Block' | '{' | 'Block -> { StmtList }' |
| 'Expr' | '(' | 'Expr -> RelExpr' |
| 'Expr' | 'char_const' | 'Expr -> RelExpr' |
| 'Expr' | 'false' | 'Expr -> RelExpr' |
| 'Expr' | 'id' | 'Expr -> RelExpr' |
| 'Expr' | 'number' | 'Expr -> RelExpr' |
| 'Expr' | 'true' | 'Expr -> RelExpr' |

```

```

| 'ExprOpt' | '(' | 'ExprOpt -> Expr' |
| 'ExprOpt' | ';' | 'ExprOpt -> epsilon' |
| 'ExprOpt' | 'char_const' | 'ExprOpt -> Expr' |
| 'ExprOpt' | 'false' | 'ExprOpt -> Expr' |
| 'ExprOpt' | 'id' | 'ExprOpt -> Expr' |
| 'ExprOpt' | 'number' | 'ExprOpt -> Expr' |
| 'ExprOpt' | 'true' | 'ExprOpt -> Expr' |
| 'Factor' | '(' | 'Factor -> ( Expr )' |
| 'Factor' | 'char_const' | 'Factor -> char_const' |
| 'Factor' | 'false' | 'Factor -> false' |
| 'Factor' | 'id' | 'Factor -> id FactorAfter' |
| 'Factor' | 'number' | 'Factor -> number' |
| 'Factor' | 'true' | 'Factor -> true' |
| 'FactorAfter' | '!=' | 'FactorAfter -> epsilon' |
| 'FactorAfter' | '%' | 'FactorAfter -> epsilon' |
| 'FactorAfter' | '(' | 'FactorAfter -> ( ArgListOpt )' |
| 'FactorAfter' | ')' | 'FactorAfter -> epsilon' |
| 'FactorAfter' | '*' | 'FactorAfter -> epsilon' |
| 'FactorAfter' | '+' | 'FactorAfter -> epsilon' |
| 'FactorAfter' | ',' | 'FactorAfter -> epsilon' |
| 'FactorAfter' | '-' | 'FactorAfter -> epsilon' |
| 'FactorAfter' | '/' | 'FactorAfter -> epsilon' |
| 'FactorAfter' | ';' | 'FactorAfter -> epsilon' |
| 'FactorAfter' | '<' | 'FactorAfter -> epsilon' |
| 'FactorAfter' | '<=' | 'FactorAfter -> epsilon' |
| 'FactorAfter' | '==' | 'FactorAfter -> epsilon' |
| 'FactorAfter' | '>' | 'FactorAfter -> epsilon' |
| 'FactorAfter' | '>=' | 'FactorAfter -> epsilon' |
| 'FactorAfter' | '[' | 'FactorAfter -> [ Expr ]' |
| 'FactorAfter' | ']' | 'FactorAfter -> epsilon' |
| 'Member' | 'boolean' | 'Member -> Type id MemberAfter' |
| 'Member' | 'char' | 'Member -> Type id MemberAfter' |
| 'Member' | 'int' | 'Member -> Type id MemberAfter' |
| 'Member' | 'void' | 'Member -> Type id MemberAfter' |
| 'MemberAfter' | '(' | 'MemberAfter -> ( ParamListOpt ) Block' |
| 'MemberAfter' | ';' | 'MemberAfter -> ;' |
| 'MemberAfter' | '[' | 'MemberAfter -> [ number ] ;' |
| 'MemberList' | 'boolean' | 'MemberList -> Member MemberList' |
| 'MemberList' | 'char' | 'MemberList -> Member MemberList' |
| 'MemberList' | 'int' | 'MemberList -> Member MemberList' |
| 'MemberList' | 'void' | 'MemberList -> Member MemberList' |
| 'MemberList' | '}' | 'MemberList -> epsilon' |
| 'MulOp' | '%' | 'MulOp -> %' |
| 'MulOp' | '*' | 'MulOp -> *' |
| 'MulOp' | '/' | 'MulOp -> /' |
| 'ParamList' | 'boolean' | 'ParamList -> Type id ParamListTail' |
| 'ParamList' | 'char' | 'ParamList -> Type id ParamListTail' |
| 'ParamList' | 'int' | 'ParamList -> Type id ParamListTail' |
| 'ParamList' | 'void' | 'ParamList -> Type id ParamListTail' |
| 'ParamListOpt' | ')' | 'ParamListOpt -> epsilon' |
| 'ParamListOpt' | 'boolean' | 'ParamListOpt -> ParamList' |
| 'ParamListOpt' | 'char' | 'ParamListOpt -> ParamList' |
| 'ParamListOpt' | 'int' | 'ParamListOpt -> ParamList' |
| 'ParamListOpt' | 'void' | 'ParamListOpt -> ParamList' |
| 'ParamListTail' | ')' | 'ParamListTail -> epsilon' |
| 'ParamListTail' | ',' | 'ParamListTail -> , Type id ParamListTail' |
| 'Program' | 'class' | 'Program -> class id { MemberList }' |
| 'RelExpr' | '(' | 'RelExpr -> AddExpr RelExprTail' |
| 'RelExpr' | 'char_const' | 'RelExpr -> AddExpr RelExprTail' |
| 'RelExpr' | 'false' | 'RelExpr -> AddExpr RelExprTail' |
| 'RelExpr' | 'id' | 'RelExpr -> AddExpr RelExprTail' |
| 'RelExpr' | 'number' | 'RelExpr -> AddExpr RelExprTail' |
| 'RelExpr' | 'true' | 'RelExpr -> AddExpr RelExprTail' |
| 'RelExprTail' | '!=' | 'RelExprTail -> RelOp AddExpr' |
| 'RelExprTail' | ')' | 'RelExprTail -> epsilon' |
| 'RelExprTail' | ',' | 'RelExprTail -> epsilon' |
| 'RelExprTail' | ';' | 'RelExprTail -> epsilon' |
| 'RelExprTail' | '<' | 'RelExprTail -> RelOp AddExpr' |
| 'RelExprTail' | '<=' | 'RelExprTail -> RelOp AddExpr' |
| 'RelExprTail' | '==' | 'RelExprTail -> RelOp AddExpr' |
| 'RelExprTail' | '>' | 'RelExprTail -> RelOp AddExpr' |
| 'RelExprTail' | '>=' | 'RelExprTail -> RelOp AddExpr' |
| 'RelExprTail' | ']' | 'RelExprTail -> epsilon' |

```

```

| 'RelOp' | '!=' | 'RelOp -> !=' |
| 'RelOp' | '<' | 'RelOp -> <' |
| 'RelOp' | '<=' | 'RelOp -> <=' |
| 'RelOp' | '==' | 'RelOp -> ==' |
| 'RelOp' | '>' | 'RelOp -> >' |
| 'RelOp' | '>=' | 'RelOp -> >=' |
| 'Stmt' | ';' | 'Stmt -> ;' |
| 'Stmt' | 'boolean' | 'Stmt -> Type id VarDeclAfter ;' |
| 'Stmt' | 'char' | 'Stmt -> Type id VarDeclAfter ;' |
| 'Stmt' | 'id' | 'Stmt -> id StmtIdTail ;' |
| 'Stmt' | 'if' | 'Stmt -> if ( Expr ) Stmt' |
| 'Stmt' | 'int' | 'Stmt -> Type id VarDeclAfter ;' |
| 'Stmt' | 'print' | 'Stmt -> print ( Expr ) ;' |
| 'Stmt' | 'return' | 'Stmt -> return ExprOpt ;' |
| 'Stmt' | 'void' | 'Stmt -> Type id VarDeclAfter ;' |
| 'Stmt' | 'while' | 'Stmt -> while ( Expr ) Stmt' |
| 'Stmt' | '{' | 'Stmt -> Block' |
| 'StmtIdTail' | '(' | 'StmtIdTail -> ( ArgListOpt )' |
| 'StmtIdTail' | '=' | 'StmtIdTail -> AssignTarget = Expr' |
| 'StmtIdTail' | '[' | 'StmtIdTail -> AssignTarget = Expr' |
| 'StmtList' | ';' | 'StmtList -> Stmt StmtList' |
| 'StmtList' | 'boolean' | 'StmtList -> Stmt StmtList' |
| 'StmtList' | 'char' | 'StmtList -> Stmt StmtList' |
| 'StmtList' | 'id' | 'StmtList -> Stmt StmtList' |
| 'StmtList' | 'if' | 'StmtList -> Stmt StmtList' |
| 'StmtList' | 'int' | 'StmtList -> Stmt StmtList' |
| 'StmtList' | 'print' | 'StmtList -> Stmt StmtList' |
| 'StmtList' | 'return' | 'StmtList -> Stmt StmtList' |
| 'StmtList' | 'void' | 'StmtList -> Stmt StmtList' |
| 'StmtList' | 'while' | 'StmtList -> Stmt StmtList' |
| 'StmtList' | '{' | 'StmtList -> Stmt StmtList' |
| 'StmtList' | '}' | 'StmtList -> epsilon' |
| 'Term' | '(' | 'Term -> Factor TermTail' |
| 'Term' | 'char_const' | 'Term -> Factor TermTail' |
| 'Term' | 'false' | 'Term -> Factor TermTail' |
| 'Term' | 'id' | 'Term -> Factor TermTail' |
| 'Term' | 'number' | 'Term -> Factor TermTail' |
| 'Term' | 'true' | 'Term -> Factor TermTail' |
| 'TermTail' | '!=' | 'TermTail -> epsilon' |
| 'TermTail' | '%' | 'TermTail -> MulOp Factor TermTail' |
| 'TermTail' | ')' | 'TermTail -> epsilon' |
| 'TermTail' | '*' | 'TermTail -> MulOp Factor TermTail' |
| 'TermTail' | '+' | 'TermTail -> epsilon' |
| 'TermTail' | ',' | 'TermTail -> epsilon' |
| 'TermTail' | '-' | 'TermTail -> epsilon' |
| 'TermTail' | '/' | 'TermTail -> MulOp Factor TermTail' |
| 'TermTail' | ';' | 'TermTail -> epsilon' |
| 'TermTail' | '<' | 'TermTail -> epsilon' |
| 'TermTail' | '<=' | 'TermTail -> epsilon' |
| 'TermTail' | '==' | 'TermTail -> epsilon' |
| 'TermTail' | '>' | 'TermTail -> epsilon' |
| 'TermTail' | '>=' | 'TermTail -> epsilon' |
| 'TermTail' | ']' | 'TermTail -> epsilon' |
| 'Type' | 'boolean' | 'Type -> boolean' |
| 'Type' | 'char' | 'Type -> char' |
| 'Type' | 'int' | 'Type -> int' |
| 'Type' | 'void' | 'Type -> void' |
| 'VarDeclAfter' | ';' | 'VarDeclAfter -> epsilon' |
| 'VarDeclAfter' | '[' | 'VarDeclAfter -> [ number ]' |

```

5 LR Parsing Table Construction

The bottom-up parser uses the original expression grammar, including left-recursive expression productions. The selected LR method is SLR(1), because the grammar's viable-prefix automaton and FOLLOW-set based reductions are sufficient for the implemented MicroJava subset without requiring canonical LR(1) item expansion.

Construction steps:

1. Augment the grammar with a new start production.
2. Build LR(0) item closures and goto transitions for terminals and non-terminals.
3. Create shift actions from terminal transitions.
4. Create reduce actions from completed items using FOLLOW sets of the production left-hand side.
5. Add accept action for the completed augmented production on end of input.
6. Store non-terminal transitions in the goto table.

5.1 LR Action and Goto Tables

```
# SLR(1) Parsing Table

Chosen method: **SLR(1)**. The grammar is small enough for an LR(0) item automaton plus FOLLOW-set
reductions, and the generated table has no conflicts.

- States: 119
- Conflicts: none

## Productions

1. 'Program -> class id { MemberList }'
2. 'MemberList -> Member MemberList'
3. 'MemberList -> epsilon'
4. 'Member -> Type id MemberAfter'
5. 'MemberAfter -> ;'
6. 'MemberAfter -> [ number ] ;'
7. 'MemberAfter -> ( ParamListOpt ) Block'
8. 'Type -> int'
9. 'Type -> char'
10. 'Type -> boolean'
11. 'Type -> void'
12. 'ParamListOpt -> ParamList'
13. 'ParamListOpt -> epsilon'
14. 'ParamList -> Type id ParamListTail'
15. 'ParamListTail -> , Type id ParamListTail'
16. 'ParamListTail -> epsilon'
17. 'Block -> { StmtList }'
18. 'StmtList -> Stmt StmtList'
19. 'StmtList -> epsilon'
20. 'Stmt -> Type id VarDeclAfter ;'
21. 'Stmt -> id StmtIdTail ;'
22. 'Stmt -> return ExprOpt ;'
23. 'Stmt -> if ( Expr ) Stmt'
24. 'Stmt -> while ( Expr ) Stmt'
25. 'Stmt -> Block'
26. 'Stmt -> ;'
27. 'Stmt -> print ( Expr ) ;'
28. 'VarDeclAfter -> [ number ]'
29. 'VarDeclAfter -> epsilon'
30. 'StmtIdTail -> AssignTarget = Expr'
31. 'StmtIdTail -> ( ArgListOpt )'
32. 'AssignTarget -> [ Expr ]'
33. 'AssignTarget -> epsilon'
34. 'ExprOpt -> Expr'
35. 'ExprOpt -> epsilon'
36. 'ArgListOpt -> ArgList'
37. 'ArgListOpt -> epsilon'
38. 'ArgList -> Expr ArgListTail'
39. 'ArgListTail -> , Expr ArgListTail'
40. 'ArgListTail -> epsilon'
41. 'RelOp -> =='
42. 'RelOp -> !='
43. 'RelOp -> <'
```

```

44. 'RelOp -> <='
45. 'RelOp -> >'
46. 'RelOp -> >='
47. 'AddOp -> +'
48. 'AddOp -> -'
49. 'MulOp -> *'
50. 'MulOp -> /'
51. 'MulOp -> %'
52. 'Factor -> id FactorAfter'
53. 'Factor -> number'
54. 'Factor -> char_const'
55. 'Factor -> true'
56. 'Factor -> false'
57. 'Factor -> ( Expr )'
58. 'FactorAfter -> ( ArgListOpt )'
59. 'FactorAfter -> [ Expr ]'
60. 'FactorAfter -> epsilon'
61. 'Expr -> Expr RelOp AddExpr'
62. 'Expr -> AddExpr'
63. 'AddExpr -> AddExpr AddOp Term'
64. 'AddExpr -> Term'
65. 'Term -> Term MulOp Factor'
66. 'Term -> Factor'

```

Action Table

State	Terminal	Action
0	'class'	's2'
1	'\$'	'acc'
2	'id'	's3'
3	'{'	's4'
4	'boolean'	's8'
4	'char'	's9'
4	'int'	's10'
4	'void'	's11'
4	'}'	'r3'
5	'boolean'	's8'
5	'char'	's9'
5	'int'	's10'
5	'void'	's11'
5	'}'	'r3'
6	'}'	's13'
7	'id'	's14'
8	'id'	'r10'
9	'id'	'r9'
10	'id'	'r8'
11	'id'	'r11'
12	'}'	'r2'
13	'\$'	'r1'
14	'('	's15'
14	','	's16'
14	'['	's18'
15	')'	'r13'
15	'boolean'	's8'
15	'char'	's9'
15	'int'	's10'
15	'void'	's11'
16	'boolean'	'r5'
16	'char'	'r5'
16	'int'	'r5'
16	'void'	'r5'
16	'}'	'r5'
17	'boolean'	'r4'
17	'char'	'r4'
17	'int'	'r4'
17	'void'	'r4'
17	'}'	'r4'
18	'number'	's22'
19	')'	'r12'
20	')'	's23'
21	'id'	's24'
22	']'	's25'

```

| 23 | '{' | 's27' |
| 24 | ')' | 'r16' |
| 24 | ',' | 's28' |
| 25 | ';' | 's30' |
| 26 | 'boolean' | 'r7' |
| 26 | 'char' | 'r7' |
| 26 | 'int' | 'r7' |
| 26 | 'void' | 'r7' |
| 26 | '}' | 'r7' |
| 27 | ';' | 's31' |
| 27 | 'boolean' | 's8' |
| 27 | 'char' | 's9' |
| 27 | 'id' | 's36' |
| 27 | 'if' | 's37' |
| 27 | 'int' | 's10' |
| 27 | 'print' | 's38' |
| 27 | 'return' | 's39' |
| 27 | 'void' | 's11' |
| 27 | 'while' | 's40' |
| 27 | '{' | 's27' |
| 27 | '}' | 'r19' |
| 28 | 'boolean' | 's8' |
| 28 | 'char' | 's9' |
| 28 | 'int' | 's10' |
| 28 | 'void' | 's11' |
| 29 | ')' | 'r14' |
| 30 | 'boolean' | 'r6' |
| 30 | 'char' | 'r6' |
| 30 | 'int' | 'r6' |
| 30 | 'void' | 'r6' |
| 30 | '}' | 'r6' |
| 31 | ';' | 'r26' |
| 31 | 'boolean' | 'r26' |
| 31 | 'char' | 'r26' |
| 31 | 'id' | 'r26' |
| 31 | 'if' | 'r26' |
| 31 | 'int' | 'r26' |
| 31 | 'print' | 'r26' |
| 31 | 'return' | 'r26' |
| 31 | 'void' | 'r26' |
| 31 | 'while' | 'r26' |
| 31 | '{' | 'r26' |
| 31 | '}' | 'r26' |
| 32 | ';' | 'r25' |
| 32 | 'boolean' | 'r25' |
| 32 | 'char' | 'r25' |
| 32 | 'id' | 'r25' |
| 32 | 'if' | 'r25' |
| 32 | 'int' | 'r25' |
| 32 | 'print' | 'r25' |
| 32 | 'return' | 'r25' |
| 32 | 'void' | 'r25' |
| 32 | 'while' | 'r25' |
| 32 | '{' | 'r25' |
| 32 | '}' | 'r25' |
| 33 | ';' | 's31' |
| 33 | 'boolean' | 's8' |
| 33 | 'char' | 's9' |
| 33 | 'id' | 's36' |
| 33 | 'if' | 's37' |
| 33 | 'int' | 's10' |
| 33 | 'print' | 's38' |
| 33 | 'return' | 's39' |
| 33 | 'void' | 's11' |
| 33 | 'while' | 's40' |
| 33 | '{' | 's27' |
| 33 | '}' | 'r19' |
| 34 | '}' | 's43' |
| 35 | 'id' | 's44' |
| 36 | '(' | 's45' |
| 36 | '=' | 'r33' |
| 36 | '[' | 's48' |
| 37 | '(' | 's49' |

```

```

| 38 | '(' | 's50' |
| 39 | '(' | 's51' |
| 39 | ';' | 'r35' |
| 39 | 'char_const' | 's57' |
| 39 | 'false' | 's58' |
| 39 | 'id' | 's59' |
| 39 | 'number' | 's60' |
| 39 | 'true' | 's61' |
| 40 | '(' | 's62' |
| 41 | 'id' | 's63' |
| 42 | '}' | 'r18' |
| 43 | ';' | 'r17' |
| 43 | 'boolean' | 'r17' |
| 43 | 'char' | 'r17' |
| 43 | 'id' | 'r17' |
| 43 | 'if' | 'r17' |
| 43 | 'int' | 'r17' |
| 43 | 'print' | 'r17' |
| 43 | 'return' | 'r17' |
| 43 | 'void' | 'r17' |
| 43 | 'while' | 'r17' |
| 43 | '{' | 'r17' |
| 43 | '}' | 'r17' |
| 44 | ';' | 'r29' |
| 44 | '[' | 's65' |
| 45 | '(' | 's51' |
| 45 | ')' | 'r37' |
| 45 | 'char_const' | 's57' |
| 45 | 'false' | 's58' |
| 45 | 'id' | 's59' |
| 45 | 'number' | 's60' |
| 45 | 'true' | 's61' |
| 46 | '=' | 's69' |
| 47 | ';' | 's70' |
| 48 | '(' | 's51' |
| 48 | 'char_const' | 's57' |
| 48 | 'false' | 's58' |
| 48 | 'id' | 's59' |
| 48 | 'number' | 's60' |
| 48 | 'true' | 's61' |
| 49 | '(' | 's51' |
| 49 | 'char_const' | 's57' |
| 49 | 'false' | 's58' |
| 49 | 'id' | 's59' |
| 49 | 'number' | 's60' |
| 49 | 'true' | 's61' |
| 50 | '(' | 's51' |
| 50 | 'char_const' | 's57' |
| 50 | 'false' | 's58' |
| 50 | 'id' | 's59' |
| 50 | 'number' | 's60' |
| 50 | 'true' | 's61' |
| 51 | '(' | 's51' |
| 51 | 'char_const' | 's57' |
| 51 | 'false' | 's58' |
| 51 | 'id' | 's59' |
| 51 | 'number' | 's60' |
| 51 | 'true' | 's61' |
| 52 | '!=' | 'r62' |
| 52 | ')' | 'r62' |
| 52 | '+' | 's75' |
| 52 | ',' | 'r62' |
| 52 | '-' | 's76' |
| 52 | ';' | 'r62' |
| 52 | '<' | 'r62' |
| 52 | '<=' | 'r62' |
| 52 | '==' | 'r62' |
| 52 | '>' | 'r62' |
| 52 | '>=' | 'r62' |
| 52 | ']' | 'r62' |
| 53 | '!=' | 's78' |
| 53 | ';' | 'r34' |
| 53 | '<' | 's79' |

```

53	'<='	's80'
53	'=='	's81'
53	'>'	's82'
53	'>='	's83'
54	','	's85'
55	'!='	'r66'
55	'%'	'r66'
55	')'	'r66'
55	'*'	'r66'
55	'+'	'r66'
55	','	'r66'
55	'-'	'r66'
55	'/'	'r66'
55	','	'r66'
55	'<'	'r66'
55	'<='	'r66'
55	'=='	'r66'
55	'>'	'r66'
55	'>='	'r66'
55	']'	'r66'
56	'!='	'r64'
56	'%'	's86'
56	')'	'r64'
56	'*'	's87'
56	'+'	'r64'
56	','	'r64'
56	'-'	'r64'
56	'/'	's88'
56	','	'r64'
56	'<'	'r64'
56	'<='	'r64'
56	'=='	'r64'
56	'>'	'r64'
56	'>='	'r64'
56	']'	'r64'
57	'!='	'r54'
57	'%'	'r54'
57	')'	'r54'
57	'*'	'r54'
57	'+'	'r54'
57	','	'r54'
57	'-'	'r54'
57	'/'	'r54'
57	','	'r54'
57	'<'	'r54'
57	'<='	'r54'
57	'=='	'r54'
57	'>'	'r54'
57	'>='	'r54'
57	']'	'r54'
58	'!='	'r56'
58	'%'	'r56'
58	')'	'r56'
58	'*'	'r56'
58	'+'	'r56'
58	','	'r56'
58	'-'	'r56'
58	'/'	'r56'
58	','	'r56'
58	'<'	'r56'
58	'<='	'r56'
58	'=='	'r56'
58	'>'	'r56'
58	'>='	'r56'
58	']'	'r56'
59	'!='	'r60'
59	'%'	'r60'
59	' ('	's90'
59	')'	'r60'
59	'*'	'r60'
59	'+'	'r60'
59	','	'r60'
59	'-'	'r60'

```

| 59 | '/' | 'r60' |
| 59 | ';' | 'r60' |
| 59 | '<' | 'r60' |
| 59 | '<=' | 'r60' |
| 59 | '==' | 'r60' |
| 59 | '>' | 'r60' |
| 59 | '>=' | 'r60' |
| 59 | '[' | 's92' |
| 59 | ']' | 'r60' |
| 60 | '!=' | 'r53' |
| 60 | '%' | 'r53' |
| 60 | ')' | 'r53' |
| 60 | '*' | 'r53' |
| 60 | '+' | 'r53' |
| 60 | ',' | 'r53' |
| 60 | '-' | 'r53' |
| 60 | '/' | 'r53' |
| 60 | ';' | 'r53' |
| 60 | '<' | 'r53' |
| 60 | '<=' | 'r53' |
| 60 | '==' | 'r53' |
| 60 | '>' | 'r53' |
| 60 | '>=' | 'r53' |
| 60 | ']' | 'r53' |
| 61 | '!=' | 'r55' |
| 61 | '%' | 'r55' |
| 61 | ')' | 'r55' |
| 61 | '*' | 'r55' |
| 61 | '+' | 'r55' |
| 61 | ',' | 'r55' |
| 61 | '-' | 'r55' |
| 61 | '/' | 'r55' |
| 61 | ';' | 'r55' |
| 61 | '<' | 'r55' |
| 61 | '<=' | 'r55' |
| 61 | '==' | 'r55' |
| 61 | '>' | 'r55' |
| 61 | '>=' | 'r55' |
| 61 | ']' | 'r55' |
| 62 | '(' | 's51' |
| 62 | 'char_const' | 's57' |
| 62 | 'false' | 's58' |
| 62 | 'id' | 's59' |
| 62 | 'number' | 's60' |
| 62 | 'true' | 's61' |
| 63 | ')' | 'r16' |
| 63 | ',' | 's28' |
| 64 | ';' | 's95' |
| 65 | 'number' | 's96' |
| 66 | ')' | 'r36' |
| 67 | ')' | 's97' |
| 68 | '!=' | 's78' |
| 68 | ')' | 'r40' |
| 68 | ',' | 's98' |
| 68 | '<' | 's79' |
| 68 | '<=' | 's80' |
| 68 | '==' | 's81' |
| 68 | '>' | 's82' |
| 68 | '>=' | 's83' |
| 69 | '(' | 's51' |
| 69 | 'char_const' | 's57' |
| 69 | 'false' | 's58' |
| 69 | 'id' | 's59' |
| 69 | 'number' | 's60' |
| 69 | 'true' | 's61' |
| 70 | ';' | 'r21' |
| 70 | 'boolean' | 'r21' |
| 70 | 'char' | 'r21' |
| 70 | 'id' | 'r21' |
| 70 | 'if' | 'r21' |
| 70 | 'int' | 'r21' |
| 70 | 'print' | 'r21' |
| 70 | 'return' | 'r21' |

```

```

| 70 | 'void' | 'r21' |
| 70 | 'while' | 'r21' |
| 70 | '{' | 'r21' |
| 70 | '}' | 'r21' |
| 71 | '!=' | 's78' |
| 71 | '<' | 's79' |
| 71 | '<=' | 's80' |
| 71 | '==' | 's81' |
| 71 | '>' | 's82' |
| 71 | '>=' | 's83' |
| 71 | ']' | 's101' |
| 72 | '!=' | 's78' |
| 72 | ')' | 's102' |
| 72 | '<' | 's79' |
| 72 | '<=' | 's80' |
| 72 | '==' | 's81' |
| 72 | '>' | 's82' |
| 72 | '>=' | 's83' |
| 73 | '!=' | 's78' |
| 73 | ')' | 's103' |
| 73 | '<' | 's79' |
| 73 | '<=' | 's80' |
| 73 | '==' | 's81' |
| 73 | '>' | 's82' |
| 73 | '>=' | 's83' |
| 74 | '!=' | 's78' |
| 74 | ')' | 's104' |
| 74 | '<' | 's79' |
| 74 | '<=' | 's80' |
| 74 | '==' | 's81' |
| 74 | '>' | 's82' |
| 74 | '>=' | 's83' |
| 75 | '(' | 'r47' |
| 75 | 'char_const' | 'r47' |
| 75 | 'false' | 'r47' |
| 75 | 'id' | 'r47' |
| 75 | 'number' | 'r47' |
| 75 | 'true' | 'r47' |
| 76 | '(' | 'r48' |
| 76 | 'char_const' | 'r48' |
| 76 | 'false' | 'r48' |
| 76 | 'id' | 'r48' |
| 76 | 'number' | 'r48' |
| 76 | 'true' | 'r48' |
| 77 | '(' | 's51' |
| 77 | 'char_const' | 's57' |
| 77 | 'false' | 's58' |
| 77 | 'id' | 's59' |
| 77 | 'number' | 's60' |
| 77 | 'true' | 's61' |
| 78 | '(' | 'r42' |
| 78 | 'char_const' | 'r42' |
| 78 | 'false' | 'r42' |
| 78 | 'id' | 'r42' |
| 78 | 'number' | 'r42' |
| 78 | 'true' | 'r42' |
| 79 | '(' | 'r43' |
| 79 | 'char_const' | 'r43' |
| 79 | 'false' | 'r43' |
| 79 | 'id' | 'r43' |
| 79 | 'number' | 'r43' |
| 79 | 'true' | 'r43' |
| 80 | '(' | 'r44' |
| 80 | 'char_const' | 'r44' |
| 80 | 'false' | 'r44' |
| 80 | 'id' | 'r44' |
| 80 | 'number' | 'r44' |
| 80 | 'true' | 'r44' |
| 81 | '(' | 'r41' |
| 81 | 'char_const' | 'r41' |
| 81 | 'false' | 'r41' |
| 81 | 'id' | 'r41' |
| 81 | 'number' | 'r41' |

```

```

| 81 | 'true' | 'r41' |
| 82 | '(' | 'r45' |
| 82 | 'char_const' | 'r45' |
| 82 | 'false' | 'r45' |
| 82 | 'id' | 'r45' |
| 82 | 'number' | 'r45' |
| 82 | 'true' | 'r45' |
| 83 | '(' | 'r46' |
| 83 | 'char_const' | 'r46' |
| 83 | 'false' | 'r46' |
| 83 | 'id' | 'r46' |
| 83 | 'number' | 'r46' |
| 83 | 'true' | 'r46' |
| 84 | '(' | 's51' |
| 84 | 'char_const' | 's57' |
| 84 | 'false' | 's58' |
| 84 | 'id' | 's59' |
| 84 | 'number' | 's60' |
| 84 | 'true' | 's61' |
| 85 | ';' | 'r22' |
| 85 | 'boolean' | 'r22' |
| 85 | 'char' | 'r22' |
| 85 | 'id' | 'r22' |
| 85 | 'if' | 'r22' |
| 85 | 'int' | 'r22' |
| 85 | 'print' | 'r22' |
| 85 | 'return' | 'r22' |
| 85 | 'void' | 'r22' |
| 85 | 'while' | 'r22' |
| 85 | '{' | 'r22' |
| 85 | '}' | 'r22' |
| 86 | '(' | 'r51' |
| 86 | 'char_const' | 'r51' |
| 86 | 'false' | 'r51' |
| 86 | 'id' | 'r51' |
| 86 | 'number' | 'r51' |
| 86 | 'true' | 'r51' |
| 87 | '(' | 'r49' |
| 87 | 'char_const' | 'r49' |
| 87 | 'false' | 'r49' |
| 87 | 'id' | 'r49' |
| 87 | 'number' | 'r49' |
| 87 | 'true' | 'r49' |
| 88 | '(' | 'r50' |
| 88 | 'char_const' | 'r50' |
| 88 | 'false' | 'r50' |
| 88 | 'id' | 'r50' |
| 88 | 'number' | 'r50' |
| 88 | 'true' | 'r50' |
| 89 | '(' | 's51' |
| 89 | 'char_const' | 's57' |
| 89 | 'false' | 's58' |
| 89 | 'id' | 's59' |
| 89 | 'number' | 's60' |
| 89 | 'true' | 's61' |
| 90 | '(' | 's51' |
| 90 | ')' | 'r37' |
| 90 | 'char_const' | 's57' |
| 90 | 'false' | 's58' |
| 90 | 'id' | 's59' |
| 90 | 'number' | 's60' |
| 90 | 'true' | 's61' |
| 91 | '!=' | 'r52' |
| 91 | '%' | 'r52' |
| 91 | ')' | 'r52' |
| 91 | '*' | 'r52' |
| 91 | '+' | 'r52' |
| 91 | ',' | 'r52' |
| 91 | '-' | 'r52' |
| 91 | '/' | 'r52' |
| 91 | ';' | 'r52' |
| 91 | '<' | 'r52' |
| 91 | '<=' | 'r52' |

```

```

| 91 | '=' | 'r52' |
| 91 | '>' | 'r52' |
| 91 | '>=' | 'r52' |
| 91 | ']' | 'r52' |
| 92 | '(' | 's51' |
| 92 | 'char_const' | 's57' |
| 92 | 'false' | 's58' |
| 92 | 'id' | 's59' |
| 92 | 'number' | 's60' |
| 92 | 'true' | 's61' |
| 93 | '!=' | 's78' |
| 93 | ')' | 's110' |
| 93 | '<' | 's79' |
| 93 | '<=' | 's80' |
| 93 | '==' | 's81' |
| 93 | '>' | 's82' |
| 93 | '>=' | 's83' |
| 94 | ')' | 'r15' |
| 95 | ';' | 'r20' |
| 95 | 'boolean' | 'r20' |
| 95 | 'char' | 'r20' |
| 95 | 'id' | 'r20' |
| 95 | 'if' | 'r20' |
| 95 | 'int' | 'r20' |
| 95 | 'print' | 'r20' |
| 95 | 'return' | 'r20' |
| 95 | 'void' | 'r20' |
| 95 | 'while' | 'r20' |
| 95 | '{' | 'r20' |
| 95 | '}' | 'r20' |
| 96 | ']' | 's111' |
| 97 | ';' | 'r31' |
| 98 | '(' | 's51' |
| 98 | 'char_const' | 's57' |
| 98 | 'false' | 's58' |
| 98 | 'id' | 's59' |
| 98 | 'number' | 's60' |
| 98 | 'true' | 's61' |
| 99 | ')' | 'r38' |
| 100 | '!=' | 's78' |
| 100 | ';' | 'r30' |
| 100 | '<' | 's79' |
| 100 | '<=' | 's80' |
| 100 | '==' | 's81' |
| 100 | '>' | 's82' |
| 100 | '>=' | 's83' |
| 101 | '=' | 'r32' |
| 102 | ';' | 's31' |
| 102 | 'boolean' | 's8' |
| 102 | 'char' | 's9' |
| 102 | 'id' | 's36' |
| 102 | 'if' | 's37' |
| 102 | 'int' | 's10' |
| 102 | 'print' | 's38' |
| 102 | 'return' | 's39' |
| 102 | 'void' | 's11' |
| 102 | 'while' | 's40' |
| 102 | '{' | 's27' |
| 103 | ';' | 's114' |
| 104 | '!=' | 'r57' |
| 104 | '%' | 'r57' |
| 104 | ')' | 'r57' |
| 104 | '*' | 'r57' |
| 104 | '+' | 'r57' |
| 104 | ',' | 'r57' |
| 104 | '-' | 'r57' |
| 104 | '/' | 'r57' |
| 104 | ';' | 'r57' |
| 104 | '<' | 'r57' |
| 104 | '<=' | 'r57' |
| 104 | '==' | 'r57' |
| 104 | '>' | 'r57' |
| 104 | '>=' | 'r57' |

```

```

| 104 | ']' | 'r57' |
| 105 | '!=' | 'r63' |
| 105 | '%' | 's86' |
| 105 | ')' | 'r63' |
| 105 | '*' | 's87' |
| 105 | '+' | 'r63' |
| 105 | ',' | 'r63' |
| 105 | '-' | 'r63' |
| 105 | '/' | 's88' |
| 105 | ';' | 'r63' |
| 105 | '<' | 'r63' |
| 105 | '<=' | 'r63' |
| 105 | '==' | 'r63' |
| 105 | '>' | 'r63' |
| 105 | '>=' | 'r63' |
| 105 | ']' | 'r63' |
| 106 | '!=' | 'r61' |
| 106 | ')' | 'r61' |
| 106 | '+' | 's75' |
| 106 | ',' | 'r61' |
| 106 | '-' | 's76' |
| 106 | ';' | 'r61' |
| 106 | '<' | 'r61' |
| 106 | '<=' | 'r61' |
| 106 | '==' | 'r61' |
| 106 | '>' | 'r61' |
| 106 | '>=' | 'r61' |
| 106 | ']' | 'r61' |
| 107 | '!=' | 'r65' |
| 107 | '%' | 'r65' |
| 107 | ')' | 'r65' |
| 107 | '*' | 'r65' |
| 107 | '+' | 'r65' |
| 107 | ',' | 'r65' |
| 107 | '-' | 'r65' |
| 107 | '/' | 'r65' |
| 107 | ';' | 'r65' |
| 107 | '<' | 'r65' |
| 107 | '<=' | 'r65' |
| 107 | '==' | 'r65' |
| 107 | '>' | 'r65' |
| 107 | '>=' | 'r65' |
| 107 | ']' | 'r65' |
| 108 | ')' | 's115' |
| 109 | '!=' | 's78' |
| 109 | '<' | 's79' |
| 109 | '<=' | 's80' |
| 109 | '==' | 's81' |
| 109 | '>' | 's82' |
| 109 | '>=' | 's83' |
| 109 | ']' | 's116' |
| 110 | ';' | 's31' |
| 110 | 'boolean' | 's8' |
| 110 | 'char' | 's9' |
| 110 | 'id' | 's36' |
| 110 | 'if' | 's37' |
| 110 | 'int' | 's10' |
| 110 | 'print' | 's38' |
| 110 | 'return' | 's39' |
| 110 | 'void' | 's11' |
| 110 | 'while' | 's40' |
| 110 | '{' | 's27' |
| 111 | ';' | 'r28' |
| 112 | '!=' | 's78' |
| 112 | ')' | 'r40' |
| 112 | ',' | 's98' |
| 112 | '<' | 's79' |
| 112 | '<=' | 's80' |
| 112 | '==' | 's81' |
| 112 | '>' | 's82' |
| 112 | '>=' | 's83' |
| 113 | ';' | 'r23' |
| 113 | 'boolean' | 'r23' |

```

```

| 113 | 'char' | 'r23' |
| 113 | 'id' | 'r23' |
| 113 | 'if' | 'r23' |
| 113 | 'int' | 'r23' |
| 113 | 'print' | 'r23' |
| 113 | 'return' | 'r23' |
| 113 | 'void' | 'r23' |
| 113 | 'while' | 'r23' |
| 113 | '{' | 'r23' |
| 113 | '}' | 'r23' |
| 114 | ';' | 'r27' |
| 114 | 'boolean' | 'r27' |
| 114 | 'char' | 'r27' |
| 114 | 'id' | 'r27' |
| 114 | 'if' | 'r27' |
| 114 | 'int' | 'r27' |
| 114 | 'print' | 'r27' |
| 114 | 'return' | 'r27' |
| 114 | 'void' | 'r27' |
| 114 | 'while' | 'r27' |
| 114 | '{' | 'r27' |
| 114 | '}' | 'r27' |
| 115 | '!=' | 'r58' |
| 115 | '%' | 'r58' |
| 115 | ')' | 'r58' |
| 115 | '*' | 'r58' |
| 115 | '+' | 'r58' |
| 115 | ',' | 'r58' |
| 115 | '-' | 'r58' |
| 115 | '/' | 'r58' |
| 115 | ';' | 'r58' |
| 115 | '<' | 'r58' |
| 115 | '<=' | 'r58' |
| 115 | '==' | 'r58' |
| 115 | '>' | 'r58' |
| 115 | '>=' | 'r58' |
| 115 | ']' | 'r58' |
| 116 | '!=' | 'r59' |
| 116 | '%' | 'r59' |
| 116 | ')' | 'r59' |
| 116 | '*' | 'r59' |
| 116 | '+' | 'r59' |
| 116 | ',' | 'r59' |
| 116 | '-' | 'r59' |
| 116 | '/' | 'r59' |
| 116 | ';' | 'r59' |
| 116 | '<' | 'r59' |
| 116 | '<=' | 'r59' |
| 116 | '==' | 'r59' |
| 116 | '>' | 'r59' |
| 116 | '>=' | 'r59' |
| 116 | ']' | 'r59' |
| 117 | ';' | 'r24' |
| 117 | 'boolean' | 'r24' |
| 117 | 'char' | 'r24' |
| 117 | 'id' | 'r24' |
| 117 | 'if' | 'r24' |
| 117 | 'int' | 'r24' |
| 117 | 'print' | 'r24' |
| 117 | 'return' | 'r24' |
| 117 | 'void' | 'r24' |
| 117 | 'while' | 'r24' |
| 117 | '{' | 'r24' |
| 117 | '}' | 'r24' |
| 118 | ')' | 'r39' |

```

Goto Table

```

| State | Non-terminal | Goto |
|----|---|---:|
| 0 | 'Program' | 1 |
| 4 | 'Member' | 5 |
| 4 | 'MemberList' | 6 |

```

```

| 4 | 'Type' | 7 |
| 5 | 'Member' | 5 |
| 5 | 'MemberList' | 12 |
| 5 | 'Type' | 7 |
| 14 | 'MemberAfter' | 17 |
| 15 | 'ParamList' | 19 |
| 15 | 'ParamListOpt' | 20 |
| 15 | 'Type' | 21 |
| 23 | 'Block' | 26 |
| 24 | 'ParamListTail' | 29 |
| 27 | 'Block' | 32 |
| 27 | 'Stmt' | 33 |
| 27 | 'StmtList' | 34 |
| 27 | 'Type' | 35 |
| 28 | 'Type' | 41 |
| 33 | 'Block' | 32 |
| 33 | 'Stmt' | 33 |
| 33 | 'StmtList' | 42 |
| 33 | 'Type' | 35 |
| 36 | 'AssignTarget' | 46 |
| 36 | 'StmtIdTail' | 47 |
| 39 | 'AddExpr' | 52 |
| 39 | 'Expr' | 53 |
| 39 | 'ExprOpt' | 54 |
| 39 | 'Factor' | 55 |
| 39 | 'Term' | 56 |
| 44 | 'VarDeclAfter' | 64 |
| 45 | 'AddExpr' | 52 |
| 45 | 'ArgList' | 66 |
| 45 | 'ArgListOpt' | 67 |
| 45 | 'Expr' | 68 |
| 45 | 'Factor' | 55 |
| 45 | 'Term' | 56 |
| 48 | 'AddExpr' | 52 |
| 48 | 'Expr' | 71 |
| 48 | 'Factor' | 55 |
| 48 | 'Term' | 56 |
| 49 | 'AddExpr' | 52 |
| 49 | 'Expr' | 72 |
| 49 | 'Factor' | 55 |
| 49 | 'Term' | 56 |
| 50 | 'AddExpr' | 52 |
| 50 | 'Expr' | 73 |
| 50 | 'Factor' | 55 |
| 50 | 'Term' | 56 |
| 51 | 'AddExpr' | 52 |
| 51 | 'Expr' | 74 |
| 51 | 'Factor' | 55 |
| 51 | 'Term' | 56 |
| 52 | 'AddOp' | 77 |
| 53 | 'RelOp' | 84 |
| 56 | 'MulOp' | 89 |
| 59 | 'FactorAfter' | 91 |
| 62 | 'AddExpr' | 52 |
| 62 | 'Expr' | 93 |
| 62 | 'Factor' | 55 |
| 62 | 'Term' | 56 |
| 63 | 'ParamListTail' | 94 |
| 68 | 'ArgListTail' | 99 |
| 68 | 'RelOp' | 84 |
| 69 | 'AddExpr' | 52 |
| 69 | 'Expr' | 100 |
| 69 | 'Factor' | 55 |
| 69 | 'Term' | 56 |
| 71 | 'RelOp' | 84 |
| 72 | 'RelOp' | 84 |
| 73 | 'RelOp' | 84 |
| 74 | 'RelOp' | 84 |
| 77 | 'Factor' | 55 |
| 77 | 'Term' | 105 |
| 84 | 'AddExpr' | 106 |
| 84 | 'Factor' | 55 |
| 84 | 'Term' | 56 |

```

```

| 89 | 'Factor' | 107 |
| 90 | 'AddExpr' | 52 |
| 90 | 'ArgList' | 66 |
| 90 | 'ArgListOpt' | 108 |
| 90 | 'Expr' | 68 |
| 90 | 'Factor' | 55 |
| 90 | 'Term' | 56 |
| 92 | 'AddExpr' | 52 |
| 92 | 'Expr' | 109 |
| 92 | 'Factor' | 55 |
| 92 | 'Term' | 56 |
| 93 | 'RelOp' | 84 |
| 98 | 'AddExpr' | 52 |
| 98 | 'Expr' | 112 |
| 98 | 'Factor' | 55 |
| 98 | 'Term' | 56 |
| 100 | 'RelOp' | 84 |
| 102 | 'Block' | 32 |
| 102 | 'Stmt' | 113 |
| 102 | 'Type' | 35 |
| 105 | 'MulOp' | 89 |
| 106 | 'AddOp' | 77 |
| 109 | 'RelOp' | 84 |
| 110 | 'Block' | 32 |
| 110 | 'Stmt' | 117 |
| 110 | 'Type' | 35 |
| 112 | 'ArgListTail' | 118 |
| 112 | 'RelOp' | 84 |

## LR(0) Item Sets

### I0
- 'Program -> . class id { MemberList }'
- 'Program' -> . Program'

### I1
- 'Program' -> Program .'

### I2
- 'Program -> class . id { MemberList }'

### I3
- 'Program -> class id . { MemberList }'

### I4
- 'Member -> . Type id MemberAfter'
- 'MemberList -> .'
- 'MemberList -> . Member MemberList'
- 'Program -> class id { . MemberList }'
- 'Type -> . boolean'
- 'Type -> . char'
- 'Type -> . int'
- 'Type -> . void'

### I5
- 'Member -> . Type id MemberAfter'
- 'MemberList -> .'
- 'MemberList -> . Member MemberList'
- 'MemberList -> Member . MemberList'
- 'Type -> . boolean'
- 'Type -> . char'
- 'Type -> . int'
- 'Type -> . void'

### I6
- 'Program -> class id { MemberList . }'

### I7
- 'Member -> Type . id MemberAfter'

### I8
- 'Type -> boolean .'

```

```

### I9
- 'Type -> char .'

### I10
- 'Type -> int .'

### I11
- 'Type -> void .'

### I12
- 'MemberList -> Member MemberList .'

### I13
- 'Program -> class id { MemberList } .'

### I14
- 'Member -> Type id . MemberAfter'
- 'MemberAfter -> . ( ParamListOpt ) Block'
- 'MemberAfter -> . ;'
- 'MemberAfter -> . [ number ] ;'

### I15
- 'MemberAfter -> ( . ParamListOpt ) Block'
- 'ParamList -> . Type id ParamListTail'
- 'ParamListOpt -> .'
- 'ParamListOpt -> . ParamList'
- 'Type -> . boolean'
- 'Type -> . char'
- 'Type -> . int'
- 'Type -> . void'

### I16
- 'MemberAfter -> ; .'

### I17
- 'Member -> Type id MemberAfter .'

### I18
- 'MemberAfter -> [ . number ] ;'

### I19
- 'ParamListOpt -> ParamList .'

### I20
- 'MemberAfter -> ( ParamListOpt . ) Block'

### I21
- 'ParamList -> Type . id ParamListTail'

### I22
- 'MemberAfter -> [ number . ] ;'

### I23
- 'Block -> . { StmtList }'
- 'MemberAfter -> ( ParamListOpt ) . Block'

### I24
- 'ParamList -> Type id . ParamListTail'
- 'ParamListTail -> .'
- 'ParamListTail -> . , Type id ParamListTail'

### I25
- 'MemberAfter -> [ number ] . ;'

### I26
- 'MemberAfter -> ( ParamListOpt ) Block .'

### I27
- 'Block -> . { StmtList }'
- 'Block -> { . StmtList }'
- 'Stmt -> . ;'
- 'Stmt -> . Block'
- 'Stmt -> . Type id VarDeclAfter ;'

```

```

- 'Stmt -> . id StmtIdTail ;'
- 'Stmt -> . if ( Expr ) Stmt'
- 'Stmt -> . print ( Expr ) ;'
- 'Stmt -> . return ExprOpt ;'
- 'Stmt -> . while ( Expr ) Stmt'
- 'StmtList -> .'
- 'StmtList -> . Stmt StmtList'
- 'Type -> . boolean'
- 'Type -> . char'
- 'Type -> . int'
- 'Type -> . void'

### I28
- 'ParamListTail -> , . Type id ParamListTail'
- 'Type -> . boolean'
- 'Type -> . char'
- 'Type -> . int'
- 'Type -> . void'

### I29
- 'ParamList -> Type id ParamListTail .'

### I30
- 'MemberAfter -> [ number ] ; .'

### I31
- 'Stmt -> ; .'

### I32
- 'Stmt -> Block .'

### I33
- 'Block -> . { StmtList }'
- 'Stmt -> . ;'
- 'Stmt -> . Block'
- 'Stmt -> . Type id VarDeclAfter ;'
- 'Stmt -> . id StmtIdTail ;'
- 'Stmt -> . if ( Expr ) Stmt'
- 'Stmt -> . print ( Expr ) ;'
- 'Stmt -> . return ExprOpt ;'
- 'Stmt -> . while ( Expr ) Stmt'
- 'StmtList -> .'
- 'StmtList -> . Stmt StmtList'
- 'StmtList -> Stmt . StmtList'
- 'Type -> . boolean'
- 'Type -> . char'
- 'Type -> . int'
- 'Type -> . void'

### I34
- 'Block -> { StmtList . }'

### I35
- 'Stmt -> Type . id VarDeclAfter ;'

### I36
- 'AssignTarget -> .'
- 'AssignTarget -> . [ Expr ]'
- 'Stmt -> id . StmtIdTail ;'
- 'StmtIdTail -> . ( ArgListOpt )'
- 'StmtIdTail -> . AssignTarget = Expr'

### I37
- 'Stmt -> if . ( Expr ) Stmt'

### I38
- 'Stmt -> print . ( Expr ) ;'

### I39
- 'AddExpr -> . AddExpr AddOp Term'
- 'AddExpr -> . Term'
- 'Expr -> . AddExpr'
- 'Expr -> . Expr RelOp AddExpr'

```

```

- 'ExprOpt -> .'
- 'ExprOpt -> . Expr'
- 'Factor -> . ( Expr )'
- 'Factor -> . char_const'
- 'Factor -> . false'
- 'Factor -> . id FactorAfter'
- 'Factor -> . number'
- 'Factor -> . true'
- 'Stmt -> return . ExprOpt ;'
- 'Term -> . Factor'
- 'Term -> . Term MulOp Factor'

### I40
- 'Stmt -> while . ( Expr ) Stmt'

### I41
- 'ParamListTail -> , Type . id ParamListTail'

### I42
- 'StmtList -> Stmt StmtList .'

### I43
- 'Block -> { StmtList } .'

### I44
- 'Stmt -> Type id . VarDeclAfter ;'
- 'VarDeclAfter -> .'
- 'VarDeclAfter -> . [ number ]'

### I45
- 'AddExpr -> . AddExpr AddOp Term'
- 'AddExpr -> . Term'
- 'ArgList -> . Expr ArgListTail'
- 'ArgListOpt -> .'
- 'ArgListOpt -> . ArgList'
- 'Expr -> . AddExpr'
- 'Expr -> . Expr RelOp AddExpr'
- 'Factor -> . ( Expr )'
- 'Factor -> . char_const'
- 'Factor -> . false'
- 'Factor -> . id FactorAfter'
- 'Factor -> . number'
- 'Factor -> . true'
- 'StmtIdTail -> ( . ArgListOpt )'
- 'Term -> . Factor'
- 'Term -> . Term MulOp Factor'

### I46
- 'StmtIdTail -> AssignTarget . = Expr'

### I47
- 'Stmt -> id StmtIdTail . ;'

### I48
- 'AddExpr -> . AddExpr AddOp Term'
- 'AddExpr -> . Term'
- 'AssignTarget -> [ . Expr ]'
- 'Expr -> . AddExpr'
- 'Expr -> . Expr RelOp AddExpr'
- 'Factor -> . ( Expr )'
- 'Factor -> . char_const'
- 'Factor -> . false'
- 'Factor -> . id FactorAfter'
- 'Factor -> . number'
- 'Factor -> . true'
- 'Term -> . Factor'
- 'Term -> . Term MulOp Factor'

### I49
- 'AddExpr -> . AddExpr AddOp Term'
- 'AddExpr -> . Term'
- 'Expr -> . AddExpr'
- 'Expr -> . Expr RelOp AddExpr'

```

```

- 'Factor -> . ( Expr )'
- 'Factor -> . char_const'
- 'Factor -> . false'
- 'Factor -> . id FactorAfter'
- 'Factor -> . number'
- 'Factor -> . true'
- 'Stmt -> if ( . Expr ) Stmt'
- 'Term -> . Factor'
- 'Term -> . Term MulOp Factor'

### I50
- 'AddExpr -> . AddExpr AddOp Term'
- 'AddExpr -> . Term'
- 'Expr -> . AddExpr'
- 'Expr -> . Expr RelOp AddExpr'
- 'Factor -> . ( Expr )'
- 'Factor -> . char_const'
- 'Factor -> . false'
- 'Factor -> . id FactorAfter'
- 'Factor -> . number'
- 'Factor -> . true'
- 'Stmt -> print ( . Expr ) ;'
- 'Term -> . Factor'
- 'Term -> . Term MulOp Factor'

### I51
- 'AddExpr -> . AddExpr AddOp Term'
- 'AddExpr -> . Term'
- 'Expr -> . AddExpr'
- 'Expr -> . Expr RelOp AddExpr'
- 'Factor -> ( . Expr )'
- 'Factor -> . ( Expr )'
- 'Factor -> . char_const'
- 'Factor -> . false'
- 'Factor -> . id FactorAfter'
- 'Factor -> . number'
- 'Factor -> . true'
- 'Term -> . Factor'
- 'Term -> . Term MulOp Factor'

### I52
- 'AddExpr -> AddExpr . AddOp Term'
- 'AddOp -> . +'
- 'AddOp -> . -'
- 'Expr -> AddExpr .'

### I53
- 'Expr -> Expr . RelOp AddExpr'
- 'ExprOpt -> Expr .'
- 'RelOp -> . !='
- 'RelOp -> . <'
- 'RelOp -> . <='
- 'RelOp -> . =='
- 'RelOp -> . >'
- 'RelOp -> . >='

### I54
- 'Stmt -> return ExprOpt . ;'

### I55
- 'Term -> Factor .'

### I56
- 'AddExpr -> Term .'
- 'MulOp -> . %'
- 'MulOp -> . *'
- 'MulOp -> . /'
- 'Term -> Term . MulOp Factor'

### I57
- 'Factor -> char_const .'

### I58

```

```

- 'Factor -> false .'

### I59
- 'Factor -> id . FactorAfter'
- 'FactorAfter -> .'
- 'FactorAfter -> . ( ArgListOpt )'
- 'FactorAfter -> . [ Expr ]'

### I60
- 'Factor -> number .'

### I61
- 'Factor -> true .'

### I62
- 'AddExpr -> . AddExpr AddOp Term'
- 'AddExpr -> . Term'
- 'Expr -> . AddExpr'
- 'Expr -> . Expr RelOp AddExpr'
- 'Factor -> . ( Expr )'
- 'Factor -> . char_const'
- 'Factor -> . false'
- 'Factor -> . id FactorAfter'
- 'Factor -> . number'
- 'Factor -> . true'
- 'Stmt -> while ( . Expr ) Stmt'
- 'Term -> . Factor'
- 'Term -> . Term MulOp Factor'

### I63
- 'ParamListTail -> , Type id . ParamListTail'
- 'ParamListTail -> .'
- 'ParamListTail -> . , Type id ParamListTail'

### I64
- 'Stmt -> Type id VarDeclAfter . ;'

### I65
- 'VarDeclAfter -> [ . number ]'

### I66
- 'ArgListOpt -> ArgList .'

### I67
- 'StmtIdTail -> ( ArgListOpt . )'

### I68
- 'ArgList -> Expr . ArgListTail'
- 'ArgListTail -> .'
- 'ArgListTail -> . , Expr ArgListTail'
- 'Expr -> Expr . RelOp AddExpr'
- 'RelOp -> . !='
- 'RelOp -> . <'
- 'RelOp -> . <='
- 'RelOp -> . =='
- 'RelOp -> . >'
- 'RelOp -> . >='

### I69
- 'AddExpr -> . AddExpr AddOp Term'
- 'AddExpr -> . Term'
- 'Expr -> . AddExpr'
- 'Expr -> . Expr RelOp AddExpr'
- 'Factor -> . ( Expr )'
- 'Factor -> . char_const'
- 'Factor -> . false'
- 'Factor -> . id FactorAfter'
- 'Factor -> . number'
- 'Factor -> . true'
- 'StmtIdTail -> AssignTarget = . Expr'
- 'Term -> . Factor'
- 'Term -> . Term MulOp Factor'

```

```

### I70
- 'Stmt -> id StmtIdTail ; .'

### I71
- 'AssignTarget -> [ Expr . ]'
- 'Expr -> Expr . RelOp AddExpr'
- 'RelOp -> . !='
- 'RelOp -> . <'
- 'RelOp -> . <='
- 'RelOp -> . =='
- 'RelOp -> . >'
- 'RelOp -> . >='

### I72
- 'Expr -> Expr . RelOp AddExpr'
- 'RelOp -> . !='
- 'RelOp -> . <'
- 'RelOp -> . <='
- 'RelOp -> . =='
- 'RelOp -> . >'
- 'RelOp -> . >='
- 'Stmt -> if ( Expr . ) Stmt'

### I73
- 'Expr -> Expr . RelOp AddExpr'
- 'RelOp -> . !='
- 'RelOp -> . <'
- 'RelOp -> . <='
- 'RelOp -> . =='
- 'RelOp -> . >'
- 'RelOp -> . >='
- 'Stmt -> print ( Expr . ) ;'

### I74
- 'Expr -> Expr . RelOp AddExpr'
- 'Factor -> ( Expr . )'
- 'RelOp -> . !='
- 'RelOp -> . <'
- 'RelOp -> . <='
- 'RelOp -> . =='
- 'RelOp -> . >'
- 'RelOp -> . >='

### I75
- 'AddOp -> + .'

### I76
- 'AddOp -> - .'

### I77
- 'AddExpr -> AddExpr AddOp . Term'
- 'Factor -> . ( Expr )'
- 'Factor -> . char_const'
- 'Factor -> . false'
- 'Factor -> . id FactorAfter'
- 'Factor -> . number'
- 'Factor -> . true'
- 'Term -> . Factor'
- 'Term -> . Term MulOp Factor'

### I78
- 'RelOp -> != .'

### I79
- 'RelOp -> < .'

### I80
- 'RelOp -> <= .'

### I81
- 'RelOp -> == .'

### I82

```

```

- 'RelOp -> > .'

### I83
- 'RelOp -> >= .'

### I84
- 'AddExpr -> . AddExpr AddOp Term'
- 'AddExpr -> . Term'
- 'Expr -> Expr RelOp . AddExpr'
- 'Factor -> . ( Expr )'
- 'Factor -> . char_const'
- 'Factor -> . false'
- 'Factor -> . id FactorAfter'
- 'Factor -> . number'
- 'Factor -> . true'
- 'Term -> . Factor'
- 'Term -> . Term MulOp Factor'

### I85
- 'Stmt -> return ExprOpt ; .'

### I86
- 'MulOp -> % .'

### I87
- 'MulOp -> * .'

### I88
- 'MulOp -> / .'

### I89
- 'Factor -> . ( Expr )'
- 'Factor -> . char_const'
- 'Factor -> . false'
- 'Factor -> . id FactorAfter'
- 'Factor -> . number'
- 'Factor -> . true'
- 'Term -> Term MulOp . Factor'

### I90
- 'AddExpr -> . AddExpr AddOp Term'
- 'AddExpr -> . Term'
- 'ArgList -> . Expr ArgListTail'
- 'ArgListOpt -> .'
- 'ArgListOpt -> . ArgList'
- 'Expr -> . AddExpr'
- 'Expr -> . Expr RelOp AddExpr'
- 'Factor -> . ( Expr )'
- 'Factor -> . char_const'
- 'Factor -> . false'
- 'Factor -> . id FactorAfter'
- 'Factor -> . number'
- 'Factor -> . true'
- 'FactorAfter -> ( . ArgListOpt )'
- 'Term -> . Factor'
- 'Term -> . Term MulOp Factor'

### I91
- 'Factor -> id FactorAfter .'

### I92
- 'AddExpr -> . AddExpr AddOp Term'
- 'AddExpr -> . Term'
- 'Expr -> . AddExpr'
- 'Expr -> . Expr RelOp AddExpr'
- 'Factor -> . ( Expr )'
- 'Factor -> . char_const'
- 'Factor -> . false'
- 'Factor -> . id FactorAfter'
- 'Factor -> . number'
- 'Factor -> . true'
- 'FactorAfter -> [ . Expr ]'
- 'Term -> . Factor'

```

```

- 'Term -> . Term MulOp Factor'

### I93
- 'Expr -> Expr . RelOp AddExpr'
- 'RelOp -> . !='
- 'RelOp -> . <'
- 'RelOp -> . <='
- 'RelOp -> . =='
- 'RelOp -> . >'
- 'RelOp -> . >='
- 'Stmt -> while ( Expr . ) Stmt'

### I94
- 'ParamListTail -> , Type id ParamListTail .'

### I95
- 'Stmt -> Type id VarDeclAfter ; .'

### I96
- 'VarDeclAfter -> [ number . ]'

### I97
- 'StmtIdTail -> ( ArgListOpt ) .'

### I98
- 'AddExpr -> . AddExpr AddOp Term'
- 'AddExpr -> . Term'
- 'ArgListTail -> , . Expr ArgListTail'
- 'Expr -> . AddExpr'
- 'Expr -> . Expr RelOp AddExpr'
- 'Factor -> . ( Expr )'
- 'Factor -> . char_const'
- 'Factor -> . false'
- 'Factor -> . id FactorAfter'
- 'Factor -> . number'
- 'Factor -> . true'
- 'Term -> . Factor'
- 'Term -> . Term MulOp Factor'

### I99
- 'ArgList -> Expr ArgListTail .'

### I100
- 'Expr -> Expr . RelOp AddExpr'
- 'RelOp -> . !='
- 'RelOp -> . <'
- 'RelOp -> . <='
- 'RelOp -> . =='
- 'RelOp -> . >'
- 'RelOp -> . >='
- 'StmtIdTail -> AssignTarget = Expr .'

### I101
- 'AssignTarget -> [ Expr ] .'

### I102
- 'Block -> . { StmtList }'
- 'Stmt -> . ;'
- 'Stmt -> . Block'
- 'Stmt -> . Type id VarDeclAfter ;'
- 'Stmt -> . id StmtIdTail ;'
- 'Stmt -> . if ( Expr ) Stmt'
- 'Stmt -> . print ( Expr ) ;'
- 'Stmt -> . return ExprOpt ;'
- 'Stmt -> . while ( Expr ) Stmt'
- 'Stmt -> if ( Expr ) . Stmt'
- 'Type -> . boolean'
- 'Type -> . char'
- 'Type -> . int'
- 'Type -> . void'

### I103
- 'Stmt -> print ( Expr ) . ;'

```

```

### I104
- 'Factor -> ( Expr ) .'

### I105
- 'AddExpr -> AddExpr AddOp Term .'
- 'MulOp -> . %'
- 'MulOp -> . *'
- 'MulOp -> . /'
- 'Term -> Term . MulOp Factor'

### I106
- 'AddExpr -> AddExpr . AddOp Term'
- 'AddOp -> . +'
- 'AddOp -> . -'
- 'Expr -> Expr RelOp AddExpr .'

### I107
- 'Term -> Term MulOp Factor .'

### I108
- 'FactorAfter -> ( ArgListOpt . )'

### I109
- 'Expr -> Expr . RelOp AddExpr'
- 'FactorAfter -> [ Expr . ]'
- 'RelOp -> . !='
- 'RelOp -> . <'
- 'RelOp -> . <='
- 'RelOp -> . =='
- 'RelOp -> . >'
- 'RelOp -> . >='

### I110
- 'Block -> . { StmtList }'
- 'Stmt -> . ;'
- 'Stmt -> . Block'
- 'Stmt -> . Type id VarDeclAfter ;'
- 'Stmt -> . id StmtIdTail ;'
- 'Stmt -> . if ( Expr ) Stmt'
- 'Stmt -> . print ( Expr ) ;'
- 'Stmt -> . return ExprOpt ;'
- 'Stmt -> . while ( Expr ) Stmt'
- 'Stmt -> while ( Expr ) . Stmt'
- 'Type -> . boolean'
- 'Type -> . char'
- 'Type -> . int'
- 'Type -> . void'

### I111
- 'VarDeclAfter -> [ number ] .'

### I112
- 'ArgListTail -> , Expr . ArgListTail'
- 'ArgListTail -> .'
- 'ArgListTail -> . , Expr ArgListTail'
- 'Expr -> Expr . RelOp AddExpr'
- 'RelOp -> . !='
- 'RelOp -> . <'
- 'RelOp -> . <='
- 'RelOp -> . =='
- 'RelOp -> . >'
- 'RelOp -> . >='

### I113
- 'Stmt -> if ( Expr ) Stmt .'

### I114
- 'Stmt -> print ( Expr ) ; .'

### I115
- 'FactorAfter -> ( ArgListOpt ) .'

```

```
### I116
- 'FactorAfter -> [ Expr ] .'

### I117
- 'Stmt -> while ( Expr ) Stmt .'

### I118
- 'ArgListTail -> , Expr ArgListTail .'
```

6 Module Descriptions

6.1 Lexical Analyzer

The lexer in `src/lexer.c` reads the source file with double buffering, tracks line and column numbers, skips whitespace and comments, recognizes keywords, identifiers, numbers, character constants, operators, and punctuation, and returns tokens on demand.

6.2 Recursive Descent Parser

The recursive descent parser in `src/recursive_descent_parser.c` implements one routine per grammar non-terminal. It uses the LL(1) transformed grammar and is also the parser connected to semantic analysis.

6.3 Predictive Parser

The predictive parser in `src/predictive_parser.c` uses an explicit stack and the LL(1) table. It includes synchronization-based recovery for syntax errors.

6.4 LR Parser

The LR parser in `src/lr_parser.c` uses a state stack, grammar-symbol stack, and the SLR(1) action/goto tables in `src/lr_tables.h`. It prints a trace showing stack contents, remaining input, and each shift/reduce/accept/error action.

6.5 Symbol Table Manager

The symbol table in `src/symbol_table.c` is hash-based and supports nested scopes through a linked scope chain. Each symbol stores name, kind, type, scope level, and declaration line. Insert, lookup, delete, and dump operations are implemented.

6.6 Error Handler

The shared error handler in `src/error_handler.c` records lexical, syntactic, and semantic errors with line and column numbers, prints clear messages, and provides a final summary.

7 Integration

The project can be built as separate lab executables or as one integrated executable. The integrated program is implemented in `src/compiler.c`. It links the lexer, recursive parser, predictive parser, LR parser, symbol table, and error handler by compiling each parser entry point under a module-specific name and dispatching from a single `compiler.exe`.

Important commands:

```
build.bat
compiler.exe lexer test\samples\hello.mj
compiler.exe recursive test\samples\hello.mj
compiler.exe predictive test\samples\hello.mj
compiler.exe lr test\samples\hello.mj
compiler.exe all test\samples\hello.mj
```

8 Sample Inputs

8.1 Valid Program: hello.mj

```
class Hello {
    void main() {
        print(1 + 2 * 3);
        return;
    }
}
```

8.2 Valid Program: factorial.mj

```
class Factorial {
    int result;

    int fact(int n) {
        int acc;
        acc = 1;
        while (n > 1) {
            acc = acc * n;
            n = n - 1;
        }
        return acc;
    }
}
```

8.3 Valid Program: arrays_and_chars.mj

```
class Data {
    char marker;
    int values[10];

    void fill() {
        int i;
        marker = 'A';
        i = 0;
        while (i < 10) {
            values[i] = i;
            i = i + 1;
        }
    }
}
```

```

    }
    return;
}
}

```

8.4 Invalid Syntax Program

```

class Broken {
    void main() {
        int x
        x = 1;
    }
}

```

8.5 Invalid Semantic Program

```

class BadSemantics {
    int x;
    int x;

    void main() {
        y = x + 1;
    }
}

```

9 Sample Outputs

9.1 All Parsers on Valid Input

```

== Recursive descent parser ==
Error summary: 0 total (0 lexical, 0 syntax, 0 semantic)
Recursive descent parser: ACCEPT

== Predictive parser ==
Error summary: 0 total (0 lexical, 0 syntax, 0 semantic)
Predictive parser: ACCEPT

== LR parser ==
SLR(1) parser trace
Grammar: original left-recursive expression grammar
Step  States      Symbols      Remaining input  Action
1      [0]      []      [class id { void id ( ) { ...]  shift 2
2      [0 2]     [class]  [id { void id ( ) { print ...]  shift 3
3      [0 2 3]   [class id]  [{ void id ( ) { print ( ...]  shift 4
4      [0 2 3 4] [class id {}]  [void id ( ) { print ( number ...]  shift 11
5      [0 2 3 4 11] [class id { void]  [id ( ) { print ( number + ...]  reduce r11: Type -> void
6      [0 2 3 4 7] [class id { Type]  [id ( ) { print ( number + ...]  shift 14
7      [0 2 3 4 7 14] [class id { Type id]  [( ) { print ( number + number ...]  shift 15
8      [0 2 3 4 7 14 15] [class id { Type id ( )]  { print ( number + number * ...]  reduce r13:
ParamListOpt -> epsilon
9      [0 2 3 4 7 14 15 20] [class id { Type id ( ParamListOpt) ]  { print ( number + number * ...]
shift 23
10     [0 2 3 4 7 14 15 20 23] [class id { Type id ( ParamListOpt )]  [{ print ( number + number * number
...]  shift 27
11     [0 2 3 4 7 14 15 20 23 27] [class id { Type id ( ParamListOpt ) {}]  [print ( number + number *
number ) ...]  shift 38
12     [0 2 3 4 7 14 15 20 23 27 38] [class id { Type id ( ParamListOpt ) { print]  [( number + number *
number ) ; ...]  shift 50
13     [0 2 3 4 7 14 15 20 23 27 38 50] [class id { Type id ( ParamListOpt ) { print ( )  [number + number
* number ) ; return ...]  shift 60
14     [0 2 3 4 7 14 15 20 23 27 38 50 60] [class id { Type id ( ParamListOpt ) { print ( number]  [+
number * number ) ; return ; ...]  reduce r53: Factor -> number

```

```

15  [0 2 3 4 7 14 15 20 23 27 38 50 55] [class id { Type id ( ParamListOpt ) { print ( Factor] [+
    number * number ) ; return ; ...] reduce r66: Term -> Factor
16  [0 2 3 4 7 14 15 20 23 27 38 50 56] [class id { Type id ( ParamListOpt ) { print ( Term] [+ number
    * number ) ; return ; ...] reduce r64: AddExpr -> Term
17  [0 2 3 4 7 14 15 20 23 27 38 50 52] [class id { Type id ( ParamListOpt ) { print ( AddExpr] [+
    number * number ) ; return ; ...] shift 75
18  [0 2 3 4 7 14 15 20 23 27 38 50 52 75] [class id { Type id ( ParamListOpt ) { print ( AddExpr +] [
    number * number ) ; return ; } ...] reduce r47: AddOp -> +
19  [0 2 3 4 7 14 15 20 23 27 38 50 52 77] [class id { Type id ( ParamListOpt ) { print ( AddExpr AddOp
    ] [number * number ) ; return ; } ...] shift 60
20  [0 2 3 4 7 14 15 20 23 27 38 50 52 77 60] [class id { Type id ( ParamListOpt ) { print ( AddExpr
    AddOp number] [* number ) ; return ; } } ...] reduce r53: Factor -> number
21  [0 2 3 4 7 14 15 20 23 27 38 50 52 77 55] [class id { Type id ( ParamListOpt ) { print ( AddExpr
    AddOp Factor] [* number ) ; return ; } } ...] reduce r66: Term -> Factor
22  [0 2 3 4 7 14 15 20 23 27 38 50 52 77 105] [class id { Type id ( ParamListOpt ) { print ( AddExpr
    AddOp Term] [* number ) ; return ; } } ...] shift 87
23  [0 2 3 4 7 14 15 20 23 27 38 50 52 77 105 87] [class id { Type id ( ParamListOpt ) { print (
    AddExpr AddOp Term *) [number ) ; return ; } } $] reduce r49: MulOp -> *
24  [0 2 3 4 7 14 15 20 23 27 38 50 52 77 105 89] [class id { Type id ( ParamListOpt ) { print (
    AddExpr AddOp Term MulOp] [number ) ; return ; } } $] shift 60
25  [0 2 3 4 7 14 15 20 23 27 38 50 52 77 105 89 60] [class id { Type id ( ParamListOpt ) { print (
    AddExpr AddOp Term MulOp number] [] ; return ; } } $] reduce r53: Factor -> number
26  [0 2 3 4 7 14 15 20 23 27 38 50 52 77 105 89 107] [class id { Type id ( ParamListOpt ) { print (
    AddExpr AddOp Term MulOp Factor] [] ; return ; } } $] reduce r65: Term -> Term
27  [0 2 3 4 7 14 15 20 23 27 38 50 52 77 105] [class id { Type id ( ParamListOpt ) { print ( AddExpr
    AddOp Term] [] ; return ; } } $] reduce r63: AddExpr -> AddExpr
28  [0 2 3 4 7 14 15 20 23 27 38 50 52] [class id { Type id ( ParamListOpt ) { print ( AddExpr] [] ;
    return ; } } $] reduce r62: Expr -> AddExpr
29  [0 2 3 4 7 14 15 20 23 27 38 50 73] [class id { Type id ( ParamListOpt ) { print ( Expr] [] ;
    return ; } } $] shift 103
30  [0 2 3 4 7 14 15 20 23 27 38 50 73 103] [class id { Type id ( ParamListOpt ) { print ( Expr )] [;
    return ; } } $] shift 114
31  [0 2 3 4 7 14 15 20 23 27 38 50 73 103 114] [class id { Type id ( ParamListOpt ) { print ( Expr )
    ;] [return ; } } $] reduce r27: Stmt -> print
32  [0 2 3 4 7 14 15 20 23 27 33] [class id { Type id ( ParamListOpt ) { Stmt] [return ; } } $] shift
    39
33  [0 2 3 4 7 14 15 20 23 27 33 39] [class id { Type id ( ParamListOpt ) { Stmt return] [; } } $]
    reduce r35: ExprOpt -> epsilon
34  [0 2 3 4 7 14 15 20 23 27 33 39 54] [class id { Type id ( ParamListOpt ) { Stmt return ExprOpt] [;
    } } $] shift 85
35  [0 2 3 4 7 14 15 20 23 27 33 39 54 85] [class id { Type id ( ParamListOpt ) { Stmt return ExprOpt
    ;] [] } } $] reduce r22: Stmt -> return
36  [0 2 3 4 7 14 15 20 23 27 33 33] [class id { Type id ( ParamListOpt ) { Stmt Stmt] [] } } $] reduce
    r19: StmtList -> epsilon
37  [0 2 3 4 7 14 15 20 23 27 33 33 42] [class id { Type id ( ParamListOpt ) { Stmt Stmt StmtList] []
    } } $] reduce r18: StmtList -> Stmt
38  [0 2 3 4 7 14 15 20 23 27 33 42] [class id { Type id ( ParamListOpt ) { Stmt StmtList] [] } } $]
    reduce r18: StmtList -> Stmt
39  [0 2 3 4 7 14 15 20 23 27 34] [class id { Type id ( ParamListOpt ) { StmtList] [] } } $] shift 43
40  [0 2 3 4 7 14 15 20 23 27 34 43] [class id { Type id ( ParamListOpt ) { StmtList ]} [] } $] reduce
    r17: Block -> {
41  [0 2 3 4 7 14 15 20 23 26] [class id { Type id ( ParamListOpt ) Block] [] } $] reduce r7:
    MemberAfter -> (
42  [0 2 3 4 7 14 17] [class id { Type id MemberAfter] [] } $] reduce r4: Member -> Type
43  [0 2 3 4 5] [class id { Member] [] } $] reduce r3: MemberList -> epsilon
44  [0 2 3 4 5 12] [class id { Member MemberList] [] } $] reduce r2: MemberList -> Member
45  [0 2 3 4 6] [class id { MemberList] [] } $] shift 13
46  [0 2 3 4 6 13] [class id { MemberList ]} [] } $] reduce r1: Program -> class
47  [0 1] [Program] [$] accept
Error summary: 0 total (0 lexical, 0 syntax, 0 semantic)
LR parser: ACCEPT

```

9.2 All Parsers on Invalid Syntax Input

```

== Recursive descent parser ==
Error summary: 4 total (0 lexical, 4 syntax, 0 semantic)
Recursive descent parser: REJECT (4 error(s))

== Predictive parser ==
Error summary: 5 total (0 lexical, 5 syntax, 0 semantic)

```

```

Predictive parser: REJECT (5 error(s))

== LR parser ==
SLR(1) parser trace
Grammar: original left-recursive expression grammar
Step  States      Symbols      Remaining input      Action
1      [0] []      [class id { void id ( ) { ...] shift 2
2      [0 2] [class] [id { void id ( ) { int ...] shift 3
3      [0 2 3] [class id] [{ void id ( ) { int id ...] shift 4
4      [0 2 3 4] [class id {}] [void id ( ) { int id id ...] shift 11
5      [0 2 3 4 11] [class id { void] [id ( ) { int id id = ...] reduce r11: Type -> void
6      [0 2 3 4 7] [class id { Type] [id ( ) { int id id = ...] shift 14
7      [0 2 3 4 7 14] [class id { Type id] [( ) { int id id = number ...] shift 15
8      [0 2 3 4 7 14 15] [class id { Type id ( ) [] { int id id = number ; ...] reduce r13: ParamListOpt
-> epsilon
9      [0 2 3 4 7 14 15 20] [class id { Type id ( ParamListOpt) [] { int id id = number ; ...] shift 23
10     [0 2 3 4 7 14 15 20 23] [class id { Type id ( ParamListOpt )] [{ int id id = number ; } ...]
shift 27
11     [0 2 3 4 7 14 15 20 23 27] [class id { Type id ( ParamListOpt ) {}] [int id id = number ; } } ...]
shift 10
12     [0 2 3 4 7 14 15 20 23 27 10] [class id { Type id ( ParamListOpt ) { int] [id id = number ; } } $]
reduce r8: Type -> int
13     [0 2 3 4 7 14 15 20 23 27 35] [class id { Type id ( ParamListOpt ) { Type] [id id = number ; } } $]
] shift 44
14     [0 2 3 4 7 14 15 20 23 27 35 44] [class id { Type id ( ParamListOpt ) { Type id] [id = number ; }
} $] error: no action for state 44 and lookahead id
15     [0 2 3 4 7 14 15 20 23 27 35 44] [class id { Type id ( ParamListOpt ) { Type id] [= number ; } } $]
] error: no action for state 44 and lookahead =
16     [0 2 3 4 7 14 15 20 23 27 35 44] [class id { Type id ( ParamListOpt ) { Type id] [number ; } } $]
error: no action for state 44 and lookahead number
17     [0 2 3 4 7 14 15 20 23 27 35 44] [class id { Type id ( ParamListOpt ) { Type id] [; } } $] reduce
r29: VarDeclAfter -> epsilon
18     [0 2 3 4 7 14 15 20 23 27 35 44 64] [class id { Type id ( ParamListOpt ) { Type id VarDeclAfter]
[; } } $] shift 95
19     [0 2 3 4 7 14 15 20 23 27 35 44 64 95] [class id { Type id ( ParamListOpt ) { Type id VarDeclAfter
;} [] } $] reduce r20: Stmt -> Type
20     [0 2 3 4 7 14 15 20 23 27 33] [class id { Type id ( ParamListOpt ) { Stmt] [] } } $] reduce r19:
StmtList -> epsilon
21     [0 2 3 4 7 14 15 20 23 27 33 42] [class id { Type id ( ParamListOpt ) { Stmt StmtList] [] } } $]
reduce r18: StmtList -> Stmt
22     [0 2 3 4 7 14 15 20 23 27 34] [class id { Type id ( ParamListOpt ) { StmtList] [] } } $] shift 43
23     [0 2 3 4 7 14 15 20 23 27 34 43] [class id { Type id ( ParamListOpt ) { StmtList []} [] } $] reduce
r17: Block -> {
24     [0 2 3 4 7 14 15 20 23 26] [class id { Type id ( ParamListOpt ) Block] [] } $] reduce r7:
MemberAfter -> (
25     [0 2 3 4 7 14 17] [class id { Type id MemberAfter] [] } $] reduce r4: Member -> Type
26     [0 2 3 4 5] [class id { Member] [] } $] reduce r3: MemberList -> epsilon
27     [0 2 3 4 5 12] [class id { Member MemberList] [] } $] reduce r2: MemberList -> Member
28     [0 2 3 4 6] [class id { MemberList] [] } $] shift 13
29     [0 2 3 4 6 13] [class id { MemberList []} []] reduce r1: Program -> class
30     [0 1] [Program] [$] accept
Error summary: 3 total (0 lexical, 3 syntax, 0 semantic)
LR parser: REJECT (3 error(s))
Syntax error at 4:5: expected ;, found IDENTIFIER 'x'
Syntax error at 4:7: expected }, found OPERATOR '='
Syntax error at 4:9: expected }, found NUMBER '1'
Syntax error at 4:10: expected end of file, found PUNCTUATION ';'
Syntax error at 4:5: no LL(1) rule for VarDeclAfter with lookahead IDENTIFIER 'x'
Syntax error at 4:7: expected ;, found OPERATOR '='
Syntax error at 4:9: no LL(1) rule for StmtList with lookahead NUMBER '1'
Syntax error at 4:10: expected }, found PUNCTUATION ';'
Syntax error at 6:1: expected $, found PUNCTUATION '}'
Syntax error at 4:5: LR error entry for state 44 and lookahead id near IDENTIFIER 'x'
Syntax error at 4:7: LR error entry for state 44 and lookahead = near OPERATOR '='
Syntax error at 4:9: LR error entry for state 44 and lookahead number near NUMBER '1'

```

9.3 Semantic Error Output

```

Error summary: 2 total (0 lexical, 0 syntax, 2 semantic)
Recursive descent parser: REJECT (2 error(s))

```

```
Semantic error at 3:7: redeclaration of 'x' in scope 0; first declared at line 2
Semantic error at 6:5: undeclared identifier 'y'
```

9.4 Symbol Table Dump

```
Symbol table dump
Name      Kind      Type      Scope  Line
main      function  void      0       2
Hello     class     class     0       1
Error summary: 0 total (0 lexical, 0 syntax, 0 semantic)
Recursive descent parser: ACCEPT
```

10 LR Trace

```
SLR(1) parser trace
Grammar: original left-recursive expression grammar
Step  States  Symbols  Remaining input  Action
1      [0]    []      [class id { void id ( ) { ...]  shift 2
2      [0 2]   [class] [id { void id ( ) { print ...]  shift 3
3      [0 2 3] [class id] [{ void id ( ) { print ( ...]  shift 4
4      [0 2 3 4] [class id {}] [void id ( ) { print ( number ...]  shift 11
5      [0 2 3 4 11] [class id { void] [id ( ) { print ( number + ...]  reduce r11: Type -> void
6      [0 2 3 4 7] [class id { Type] [id ( ) { print ( number + ...]  shift 14
7      [0 2 3 4 7 14] [class id { Type id] [( ) { print ( number + number ...]  shift 15
8      [0 2 3 4 7 14 15] [class id { Type id ( )] [{ print ( number + number * ...]  reduce r13: ParamListOpt -> epsilon
9      [0 2 3 4 7 14 15 20] [class id { Type id ( ParamListOpt) ] { print ( number + number * ...]  shift 23
10     [0 2 3 4 7 14 15 20 23] [class id { Type id ( ParamListOpt )} [{ print ( number + number * number ...]  shift 27
11     [0 2 3 4 7 14 15 20 23 27] [class id { Type id ( ParamListOpt ) {}] [print ( number + number * number ) ...]  shift 38
12     [0 2 3 4 7 14 15 20 23 27 38] [class id { Type id ( ParamListOpt ) { print] [( number + number * number ) ; ...]  shift 50
13     [0 2 3 4 7 14 15 20 23 27 38 50] [class id { Type id ( ParamListOpt ) { print ( )] [number + number * number ) ; return ...]  shift 60
14     [0 2 3 4 7 14 15 20 23 27 38 50 60] [class id { Type id ( ParamListOpt ) { print ( number] [+ number * number ) ; return ; ...]  reduce r53: Factor -> number
15     [0 2 3 4 7 14 15 20 23 27 38 50 55] [class id { Type id ( ParamListOpt ) { print ( Factor] [+ number * number ) ; return ; ...]  reduce r66: Term -> Factor
16     [0 2 3 4 7 14 15 20 23 27 38 50 56] [class id { Type id ( ParamListOpt ) { print ( Term] [+ number * number ) ; return ; ...]  reduce r64: AddExpr -> Term
17     [0 2 3 4 7 14 15 20 23 27 38 50 52] [class id { Type id ( ParamListOpt ) { print ( AddExpr] [+ number * number ) ; return ; ...]  shift 75
18     [0 2 3 4 7 14 15 20 23 27 38 50 52 75] [class id { Type id ( ParamListOpt ) { print ( AddExpr +] [ number * number ) ; return ; } ...]  reduce r47: AddOp -> +
19     [0 2 3 4 7 14 15 20 23 27 38 50 52 77] [class id { Type id ( ParamListOpt ) { print ( AddExpr AddOp] [number * number ) ; return ; } ...]  shift 60
20     [0 2 3 4 7 14 15 20 23 27 38 50 52 77 60] [class id { Type id ( ParamListOpt ) { print ( AddExpr AddOp number] [* number ) ; return ; } } ...]  reduce r53: Factor -> number
21     [0 2 3 4 7 14 15 20 23 27 38 50 52 77 55] [class id { Type id ( ParamListOpt ) { print ( AddExpr AddOp Factor] [* number ) ; return ; } } ...]  reduce r66: Term -> Factor
22     [0 2 3 4 7 14 15 20 23 27 38 50 52 77 105] [class id { Type id ( ParamListOpt ) { print ( AddExpr AddOp Term] [* number ) ; return ; } } ...]  shift 87
23     [0 2 3 4 7 14 15 20 23 27 38 50 52 77 105 87] [class id { Type id ( ParamListOpt ) { print ( AddExpr AddOp Term *} [number ) ; return ; } } $]  reduce r49: MulOp -> *
24     [0 2 3 4 7 14 15 20 23 27 38 50 52 77 105 89] [class id { Type id ( ParamListOpt ) { print ( AddExpr AddOp Term MulOp] [number ) ; return ; } } $]  shift 60
25     [0 2 3 4 7 14 15 20 23 27 38 50 52 77 105 89 60] [class id { Type id ( ParamListOpt ) { print ( AddExpr AddOp Term MulOp number] [ ] ; return ; } } $]  reduce r53: Factor -> number
26     [0 2 3 4 7 14 15 20 23 27 38 50 52 77 105 89 107] [class id { Type id ( ParamListOpt ) { print ( AddExpr AddOp Term MulOp Factor] [ ] ; return ; } } $]  reduce r65: Term -> Term
27     [0 2 3 4 7 14 15 20 23 27 38 50 52 77 105] [class id { Type id ( ParamListOpt ) { print ( AddExpr AddOp Term] [ ] ; return ; } } $]  reduce r63: AddExpr -> AddExpr
28     [0 2 3 4 7 14 15 20 23 27 38 50 52] [class id { Type id ( ParamListOpt ) { print ( AddExpr] [ ] ; return ; } } $]  reduce r62: Expr -> AddExpr
```

```

29  [0 2 3 4 7 14 15 20 23 27 38 50 73]  [class id { Type id ( ParamListOpt ) { print ( Expr]  [] ;
    return ; } } $]  shift 103
30  [0 2 3 4 7 14 15 20 23 27 38 50 73 103]  [class id { Type id ( ParamListOpt ) { print ( Expr )]  [;
    return ; } } $]  shift 114
31  [0 2 3 4 7 14 15 20 23 27 38 50 73 103 114]  [class id { Type id ( ParamListOpt ) { print ( Expr )
    ;]  [return ; } } $]  reduce r27: Stmt -> print
32  [0 2 3 4 7 14 15 20 23 27 33]  [class id { Type id ( ParamListOpt ) { Stmt]  [return ; } } $]  shift
    39
33  [0 2 3 4 7 14 15 20 23 27 33 39]  [class id { Type id ( ParamListOpt ) { Stmt return]  [; } } $]
    reduce r35: ExprOpt -> epsilon
34  [0 2 3 4 7 14 15 20 23 27 33 39 54]  [class id { Type id ( ParamListOpt ) { Stmt return ExprOpt]  [;
    } } $]  shift 85
35  [0 2 3 4 7 14 15 20 23 27 33 39 54 85]  [class id { Type id ( ParamListOpt ) { Stmt return ExprOpt
    ;]  [ ] } } $]  reduce r22: Stmt -> return
36  [0 2 3 4 7 14 15 20 23 27 33 33]  [class id { Type id ( ParamListOpt ) { Stmt Stmt]  [ ] } } $]  reduce
    r19: StmtList -> epsilon
37  [0 2 3 4 7 14 15 20 23 27 33 33 42]  [class id { Type id ( ParamListOpt ) { Stmt Stmt StmtList]  [ ]
    } } $]  reduce r18: StmtList -> Stmt
38  [0 2 3 4 7 14 15 20 23 27 33 42]  [class id { Type id ( ParamListOpt ) { Stmt StmtList]  [ ] } } $]
    reduce r18: StmtList -> Stmt
39  [0 2 3 4 7 14 15 20 23 27 34]  [class id { Type id ( ParamListOpt ) { StmtList]  [ ] } } $]  shift 43
40  [0 2 3 4 7 14 15 20 23 27 34 43]  [class id { Type id ( ParamListOpt ) { StmtList } ]  [ ] } $]  reduce
    r17: Block -> {
41  [0 2 3 4 7 14 15 20 23 26]  [class id { Type id ( ParamListOpt ) Block]  [ ] } $]  reduce r7:
    MemberAfter -> (
42  [0 2 3 4 7 14 17]  [class id { Type id MemberAfter]  [ ] } $]  reduce r4: Member -> Type
43  [0 2 3 4 5]  [class id { Member]  [ ] } $]  reduce r3: MemberList -> epsilon
44  [0 2 3 4 5 12]  [class id { Member MemberList]  [ ] } $]  reduce r2: MemberList -> Member
45  [0 2 3 4 6]  [class id { MemberList]  [ ] } $]  shift 13
46  [0 2 3 4 6 13]  [class id { MemberList } ]  [ ] } $]  reduce r1: Program -> class
47  [0 1]  [Program]  [ ] } $]  accept
LR parser: ACCEPT

```

11 Limitations

- The compiler implements front-end analysis only; it does not generate intermediate code, assembly, or bytecode.
- Semantic analysis checks declarations, redeclarations, scope nesting, and identifier usage, but it does not perform full type checking for every expression and function call.
- The LR parser reports and recovers from syntax errors by consuming invalid lookahead tokens; recovery is suitable for demonstration but not a full production-grade diagnostic system.
- The grammar is a compact MicroJava subset rather than the complete Java language.

12 References

- Aho, Lam, Sethi, and Ullman, *Compilers: Principles, Techniques, and Tools*.
- CS-471L Compiler Construction Lab handouts.
- Project grammar, parsing tables, and implementation files included in this repository.

13 Conclusion

All required project sections are complete. The final repository includes grammar artifacts, lexer, three parsers, semantic analysis, error handling, integrated executable support, sample tests, demonstration outputs, and this final report.